SCHOOL OF COMPUTATION, INFORMATION
AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Image Classification for Nazi Symbols

Zhiwei Zhang

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Image Classification for Nazi Symbols

Bildklassifikation zur Erkennung nationalsozialistischer Symbole

Author:	Zhiwei Zhang
Examiner:	Prof. Dr. Jürgen Pfeffer
Supervisor:	Daniel Matter
Submission Date:	15.07.2025

I confirm that this master’s thesis is my own work and I have documented all sources and material used.

Munich, 15.07.2025 Zhiwei Zhang

Acknowledgments

Abstract

The detection and classification of Nazi-related symbols in digital images are critical tasks in combating the spread of extremist content and supporting social science research on hate speech. This thesis investigates modern image classification methods for identifying Nazi symbols, including both binary classification (detecting whether any Nazi symbol is present) and multi-class classification (identifying the specific symbol type). A curated dataset was compiled, consisting of 99,435 non-Nazi images and 2,338 Nazi-related images spanning 22 symbol categories. Due to limited examples in some classes, several were regrouped into a consolidated *Neo-Nazi* category to improve model learnability.

Multiple Deep Learning (DL) architectures were evaluated, including You Only Look Once (YOLO), Residual Network (ResNet), EfficientNet, Vision Transformer (ViT), OpenCLIP, and Moondream. Additionally, classical machine learning models were explored using OpenCLIP visual embeddings as input features. Model performance was assessed using standard metrics such as accuracy, precision, recall, and F1-score.

Experimental results demonstrate that for the binary classification task, OpenCLIP embeddings combined with classical classifiers achieved competitive performance while requiring significantly fewer computational resources. However, in the multi-class classification task, this combination underperformed substantially compared to ViT and YOLO, with YOLO achieving the best overall performance in identifying specific symbol classes.

To support practical application, the best-performing models were integrated into an Application Programming Interface (API) service. A Continuous Integration/Continuous Delivery (CI/CD) pipeline was implemented to automate packaging and publishing of the service as a Docker image, enabling easy deployment by researchers or institutions. This work provides a lightweight and effective solution for detecting Nazi symbols in large-scale image datasets and contributes to broader efforts in content moderation and digital extremism research.

Contents

1 Introduction

1.1 Background

Social media platforms have emerged as primary channels for the dissemination of vast quantities of multimedia content, including images, text, and video. These digital streams provide a rich source of data for research in the social sciences, particularly in fields such as political extremism, regional conflict analysis, and hate speech detection. To facilitate such research, scholars often rely on web crawlers to harvest large datasets from online platforms. However, the raw data collected through these methods are typically unstructured and lack semantic annotation, requiring significant preprocessing to extract meaningful insights.

For image data in particular, classification serves as a critical preprocessing step, transforming unstructured content into labeled datasets that are suitable for both quantitative and qualitative analysis. Among the various categories of harmful or sensitive visual content, Nazi-related imagery remains especially salient. Symbols such as the swastika, once co-opted by the German National Socialist regime (1933–1945), continue to function as enduring emblems of hate and extremism. Additional symbols—such as the Schutzstaffel (SS) bolts, the Black Sun (linked to fascist occult ideologies), and the Siegrune—have been appropriated by modern neo-Nazi and far-right movements. These symbols persist across digital platforms, where they are employed to signal ideological affiliation, propagate extremist views, or incite violence. Given that many countries restrict or criminalize the display of such imagery, there is a pressing need for automated detection tools to support content moderation and policy enforcement.

1.2 Problem Statement

The ability to automatically detect and classify Nazi symbols in images is critical for researchers studying the dynamics of extremism, as well as for platforms and authorities aiming to curb the spread of hate speech. However, commercially available image classification services that offer such functionality are often proprietary and prohibitively expensive. This presents a significant barrier for researchers and civil society actors operating under limited budgets.

To address this gap, there is a need for an accessible, transparent, and effective image classification system tailored specifically for the detection of Nazi symbols. Such a system should be capable of both binary classification (detecting the presence or absence of Nazi imagery) and multi-class classification (identifying the specific symbol present). Furthermore, it must be lightweight and adaptable, with the ability to operate efficiently on modest hardware resources such as a single-GPU workstation.

1.3 Objectives

This research aims to design, implement, and evaluate an efficient and robust image classification framework for the automatic detection and categorization of Nazi symbols. The specific objectives of the study are as follows:

- Construct a curated dataset of approximately 100,000 images, encompassing both Nazi-related and non-Nazi imagery.
- Annotate and structure the dataset to support both binary and multi-class classification tasks.
- Investigate a diverse range of DL and multimodal models—including Convolutional Neural Network (CNN)s, ViT, and OpenCLIP embeddings—paired with classical Machine Learning (ML) algorithms.
- Evaluate model performance using standard classification metrics, and analyze the comparative strengths and limitations of each approach.
- Develop an API service to enable model inference via HTTP requests, thereby facilitating real-world deployment.

While this study is bounded by constraints in data coverage and computational resources, its overarching aim is to propose a generalizable methodology that can be scaled and adapted to broader categories of extremist symbols and larger datasets.

1.4 Scope and Limitations

The visual symbols associated with Nazi and neo-Nazi ideologies are numerous and varied, both historically and in contemporary usage. Given this diversity, it is infeasible to construct a comprehensive dataset that captures all such symbols. Instead, this study focuses on a representative subset of the most visually distinctive and commonly encountered symbols. Specifically, the following classes are included: *Black Sun*, *Flash and Circle*, *Broken Sun Cross*, *The Happy Merchant*, *Hitler*, *Nazi Salute*, *Atomwaffen Division Emblem*, *Blood & Honor Emblem*, *Celtic Cross*, *Combat 18 Emblem*, *Golden Dawn*, *Hammerskins Emblem*, *Identitarian Movement Emblem*, *Kolovrat*, *National Rebirth of Poland Emblem*, *Popular Front Emblem*, *Yellow Badge*, *Siegrune*, *SS Skull*, *Sturmabteilung Emblem*, *Swastika*, and *Wolfsangel*.

The classification system is designed to support two core tasks:

1. **Binary Classification:** Determining whether a given image contains any Nazi-related symbol.

2. **Multi-Class Classification:** Identifying which specific symbol (among 13 predefined categories) is present in an image labeled as Nazi-related.

In addition to dataset limitations, the study is constrained by hardware availability. All experiments were conducted on a single GPU with 16 GB of memory, which limited the scale of hyperparameter tuning and model ensembling. Consequently, priority was given to computationally efficient architectures and training procedures.

Despite these constraints, this work seeks to demonstrate the feasibility of developing effective, accessible, and ethically responsible tools for the detection of extremist visual content. It is intended to support researchers, content moderators, and policymakers in their efforts to monitor and counteract the dissemination of hate imagery online.

1.5 Thesis Structure

This thesis is organized as follows:

- **Chapter 2: Literature Review** – This chapter surveys existing research on Nazi symbol detection and classification, emphasizing methodological trends, key findings, and identifying research gaps addressed in this work.

- **Chapter 3: Data and Model Development Methodology** – This chapter outlines the data collection process, dataset composition, preprocessing techniques, and the classification algorithms employed. It also describes the model training process, evaluation metrics, and justifies the chosen methodologies.

- **Chapter 4: Ethical Considerations** – We explore the ethical implications of working with sensitive content, including data privacy, the potential for misuse, and the broader responsibilities associated with automated hate symbol detection.

- **Chapter 5: Results** – This chapter presents the outcomes of our experiments, providing both quantitative and qualitative analyses of model performance on binary and multi-class classification tasks.

- **Chapter 6: Discussion and Analysis** – We interpret the results, analyze their implications in light of the research objectives, compare our findings with prior work, and highlight the system's limitations and areas for improvement.
- **Chapter 7: Model Deployment and API Service** – This chapter describes the deployment strategy for the best-performing models, including the architecture of the API service, the CI/CD pipeline, and details of the available endpoints.
- **Chapter 8: Conclusion and Future Work** – The final chapter summarizes the contributions of this thesis, reflects on key insights gained, and outlines directions for future research in the field of Nazi symbol classification.

2 Literature Review

The classification of Nazi symbols in images is a highly specialized task situated at the intersection of computer vision, hate speech detection, few-shot learning, and ethical Artifical Intelligence (AI) deployment. Owing to the diverse and sensitive visual characteristics of such symbols, this problem introduces unique challenges, including the need for robust visual representations, limited annotated data, and open-set recognition capabilities.

2.1 Visual Symbol and Logo Recognition

Foundational work in logo and symbol recognition has informed methods for visual hate symbol detection. Iandola et al. [Ian+15] demonstrated that CNNs are effective at recognizing logos in natural scenes, which closely parallels the challenges of detecting hate symbols embedded in visually complex or cluttered environments. Expanding on scalability and intra-class variability, Wang et al. [Wan+19] proposed Logo-2K+, a large-scale logo dataset comprising over 2,000 categories, highlighting the importance of generalization across visually similar but distinct symbol classes.

Real-world applications of Nazi symbol recognition must also contend with stylistic variations, modifications, and unseen symbol variants. Tüzkö et al. [Tüz+17] addressed these issues through open-set logo detection techniques, enabling models to differentiate between known and unknown logos. This capability is essential for the classification of hate symbols, which often appear in modified or contextually obscured forms.

2.2 Hate Content Detection in Social Media and Memes

Hate symbol recognition increasingly intersects with multimodal content analysis, especially in memes and online extremist media. Belete and Alitasb [BA25] developed a deelp learning pipeline for detecting offensive Amharic-language memes using Bidirectional Long Short-Term Memory (BiLSTM) and dense networks. Although language-specific, their work demonstrates the potential of combining textual and visual cues for hate content detection—a key consideration for memes that integrate Nazi symbols with provocative slogans or captions.

Das Bhattacharjee et al. [DTP19] adopted a context-aware active learning framework to identify malicious content on social platforms. Their results underscore the importance of contextual information—particularly relevant when hate symbols are embedded in complex visual narratives or are visually subtle. In parallel, Scrivens et al. [Scr+21] examined how violent right-wing extremist groups use imagery in online spaces, offering sociological insights that reinforce the need for comprehensive visual detection approaches capable of capturing both overt and covert symbolism.

2.3 Deep Learning Models for Visual Classification

Modern image classification systems largely rely on deep neural networks. AlexNet [KSH12] marked a significant milestone in DL, introducing a convolutional framework that enabled large-scale visual classification. Subsequent advancements such as Visual Geometry Group Network (VGGNet) [SZ15] demonstrated that deeper networks can capture finer-grained visual patterns—an important feature when symbols are small, degraded, or partially obscured.

More recently, the ViT architecture introduced by Dosovitskiy et al. [Dos+21] replaced convolutions with self-attention mechanisms, enabling models to process global image context. This is particularly beneficial for recognizing Nazi symbols that may appear in cluttered or visually dense settings, such as posters, flags, or memes.

2.4 Zero-Shot and Few-Shot Symbol Recognition

A persistent challenge in Nazi symbol classification is the scarcity of labeled training data, especially for less common or region-specific variations. In such cases, zero-shot and few-shot learning methods become essential. Xian et al. [Xia+19] systematically benchmarked Zero-Shot Learning (ZSL) approaches, underscoring the trade-offs between generalization and domain adaptation.

Contrastive Language-Image Pre-training (CLIP), introduced by Radford et al. [Rad+21], leverages contrastive learning to align image and text representations, allowing for zero-shot classification based on descriptive prompts. This modality is especially promising for detecting evolving or rarely seen Nazi symbols. DiffCLIP [Zha+24] extends this work by enabling prompt-based fine-tuning, making it suitable for few-shot scenarios where only minimal labeled data are available.

Relation Network (RN) [Sun+18] offer an alternative few-shot approach by classifying unseen categories based on feature similarity. These methods are highly relevant in contexts where limited annotated datasets of hate symbols must be leveraged for generalization to new symbol types or stylized variants.

2.5 Dataset Transparency and Ethical Considerations

The sensitive nature of Nazi symbol detection necessitates responsible and transparent AI practices. Gebru et al. [Geb+21] introduced the concept of "datasheets for datasets," advocating for comprehensive documentation to promote dataset transparency, provenance tracking, and bias mitigation. Such practices are vital when working with hate-related or politically sensitive imagery.

Additionally, Shahriari [SS17] emphasized ethically aligned design in AI systems, encouraging practitioners to prioritize human rights, safety, and fairness. These principles are particularly salient for Nazi symbol classifiers, where the risk of misuse, false positives, or sociopolitical harm must be carefully mitigated through responsible design and deployment.

3 Data and Model Development Methodology

This chapter outlines the methodology adopted for developing and evaluating the proposed Nazi symbol classification system. It begins with a detailed overview of the dataset construction process, including the sources of imagery and structuring strategies tailored for both binary and multi-class classification tasks. The chapter then describes the taxonomy of Nazi symbols included in the dataset, offering historical and visual context for each symbol class. A summary of the distribution of these symbols is provided in Table 3.2, while the relative proportions of the binary and multi-class datasets are visualized in Figure 3.3 and Figure 3.4, respectively. Subsequently, the ML and DL models used are introduced, followed by the preprocessing pipeline, training procedures, and hardware configurations. Finally, the evaluation metrics used to assess model performance are presented, forming the foundation for the experimental results in the subsequent chapter.

3.1 Dataset Construction and Symbol Taxonomy

To develop a reliable image classification system capable of identifying Nazi symbols in diverse visual contexts, a custom dataset was curated from multiple public sources. This dataset supports both binary and multi-class classification tasks by incorporating a broad range of Nazi and non-Nazi imagery, annotated and structured with attention to historical accuracy, visual clarity, and class balance.

3.1.1 Data Sources

The dataset was assembled from two primary platforms:

- **Roboflow**: Provided labeled datasets containing images of known Nazi symbols such as the swastika, SS bolts, and Black Sun. These were selected for their relevance, annotation quality, and visual variability, forming the positive class.
- **Kaggle**: Contributed general-purpose datasets consisting of images unrelated to extremism, including scenes from daily life, commercial logos, and various public domains. These constituted the negative class, essential for training models to distinguish hate symbols from benign visual content.

All datasets were obtained under open-use or research-oriented licenses, with careful review of usage terms and ethical considerations.

3.1.2 Preprocessing and Quality Control

To ensure dataset reliability and suitability for training, the following preprocessing steps were applied:

- **Manual Verification**: All symbol annotations were manually reviewed to verify label correctness and remove noise.
- **Symbol Taxonomy Alignment**: Nazi symbols were categorized based on visual features and historical context, referencing established taxonomies from the Anti-Defamation League (ADL) [13] and academic literature on extremist iconography.
- **Deduplication**: Duplicate and near-duplicate images were removed to reduce overfitting and prevent data leakage.
- **Partitioning**: The dataset was randomly split into 70% training, 15% validation, and 15% test sets, ensuring no overlap of symbol instances across splits.

A summary of all datasets is provided in Table 3.1, and representative examples from the positive and negative classes are shown in Figure 3.1.

(a) Sample images containing Nazi symbols

(b) Sample images not containing Nazi symbols

Figure 3.1: Examples from the positive and negative classes

Platform	Dataset Name / Description	Purpose	Notes
Roboflow	detect Dataset [mar23]	Positive Class	Swastika and related Nazi symbols
Roboflow	swastika Dataset (imgann) [img23a]	Positive Class	Swastika symbol detection
Roboflow	swastika Dataset (Darja Nechaeva) [Nec22]	Positive Class	Historical swastika variations
Roboflow	swastika_detect Dataset [Inc23]	Positive Class	Swastika detection for moderation
Roboflow	symbols Dataset [Vol23]	Positive Class	Includes general and hate symbols
Roboflow	violent object Dataset [img23b]	Positive Class	Hate symbols and violent imagery
Roboflow	Swastiki_Net Dataset [He624]	Positive Class	Swastika symbol classification
Roboflow	Content Moderation Dataset [Tec24]	Positive Class	Multiple harmful or extremist symbols
Roboflow	Swastika_Detection Dataset [Kav22]	Positive Class	Swastikas in different contexts
Kaggle	100 Sports Image Classification [Ger23]	Negative Class	Neutral sports images
Kaggle	Car vs Bike Classification Dataset [Dee22]	Negative Class	Background class diversity
Kaggle	People Clothing Segmentation [Lak21]	Negative Class	Background objects, neutral content
Kaggle	Fruits and Vegetables Dataset [SET22]	Negative Class	For reducing false positives
Kaggle	Animal Image Dataset [BAN22]	Negative Class	Animal images for neutral comparison
Kaggle	Kids and Adults Detection [GIH21]	Negative Class	Human figures, non-extremist
Kaggle	SCOOBY DOO Classification [DEL25]	Negative Class	Cartoon data for variance
Kaggle	Low Light Animals [Sar25]	Negative Class	Low-light conditions for robustness
Kaggle	AI vs Human-Generated Images [Shu25]	Negative Class	Diverse image source simulation

Table 3.1: Summary of datasets used in this thesis for training, validation, and testing.

3.1.3 Symbol Taxonomy and Description

The collected dataset includes 22 distinct symbols associated with Nazi ideology, grouped based on their historical origin, contemporary usage, and visual form. These symbols were selected due to their recognized use in Nazi Germany and/or modern neo-Nazi and white supremacist movements. Figure 3.2 provides a visual overview.

Below is a concise description of each symbol class included in the dataset:

- **Black Sun**: A circular symbol composed of twelve radial Sig runes, associated with SS mysticism and adopted by neo-Nazi groups as an esoteric representation of Aryan heritage.
- **Flash and Circle**: A lightning bolt within a circle, originally used by the British Union of Fascists; later adopted by various white supremacist groups.
- **Broken Sun Cross**: A variation of the ancient sun cross with swastika-like arms; historically linked to Nazi-affiliated movements and Waffen-SS units.
- **Happy Merchant**: A racist caricature of a Jewish man, originating from hate propaganda and widely circulated in white supremacist memes.
- **Hitler**: Images of Adolf Hitler, representing the central figure of Nazi ideology.
- **Nazi Salute**: The Sieg Heil gesture used to demonstrate allegiance to Hitler; currently banned in several countries.

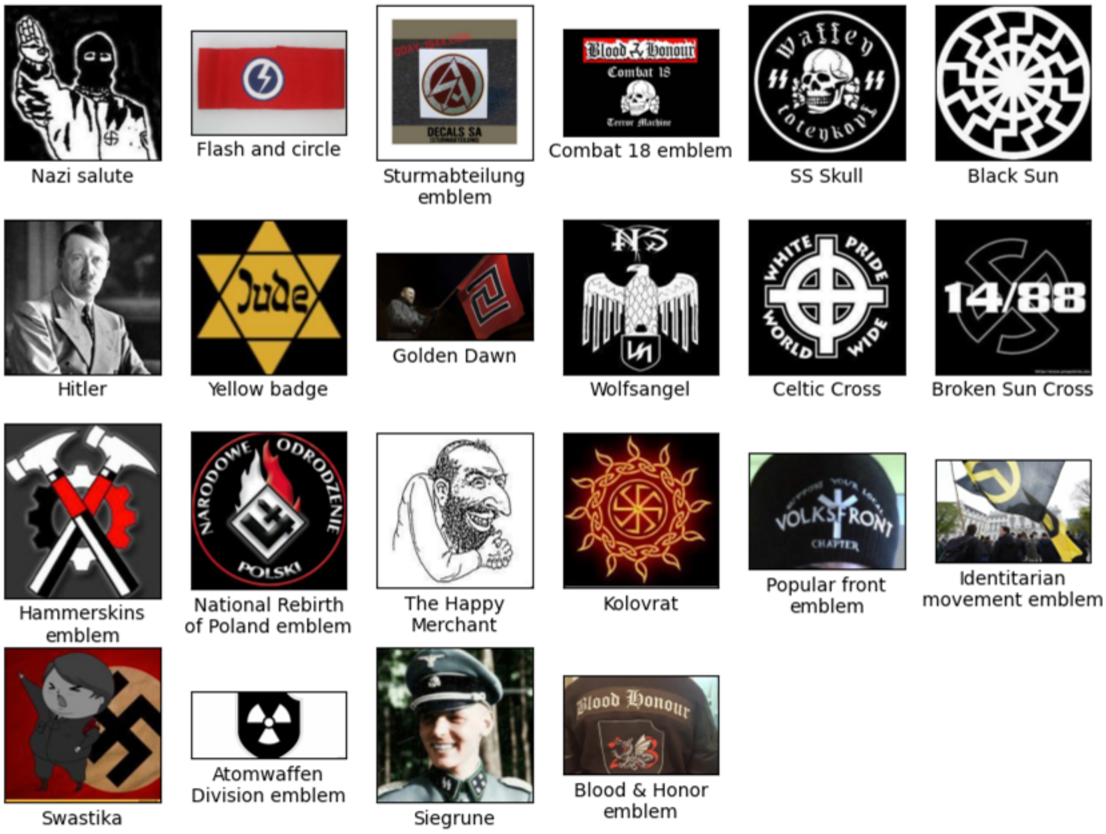


Figure 3.2: Example images from 22 Nazi symbol classes

- **Yellow Badge**: A yellow Star of David used to identify Jews during the Holocaust, symbolizing systemic persecution.
- **Siegrune**: An SS lightning-bolt rune adapted from Germanic runes, widely used by the Nazi Schutzstaffel.
- **SS Skull (Totenkopf)**: A skull symbol signifying loyalty unto death, worn by SS personnel, especially in concentration camps.
- **Sturmabteilung (SA) Emblem**: Emblem of the Nazi Party's original paramilitary wing, representing early violent enforcement of Nazi ideology.
- **Swastika**: A symbol appropriated by the Nazis to signify racial purity and nationalism; now the most recognizable icon of hate worldwide.

- **Wolfsangel**: Originally a medieval glyph, later adopted by Nazi divisions and modern neo-Nazi movements.
- **Atomwaffen Division Emblem**: A stylized nuclear symbol used by a neo-Nazi terrorist group advocating for white supremacist accelerationism.
- **Blood and Honour Emblem**: Derived from the Hitler Youth motto "Blut und Ehre"; used by an international neo-Nazi network promoting white nationalist music.
- **Celtic Cross**: While historically religious, its simplified form is now a common white nationalist symbol.
- **Combat 18 Emblem**: Insignia of a violent neo-Nazi group, with "18" representing Adolf Hitler by numerical substitution.
- **Golden Dawn**: Emblem of a Greek far-right political party with known neo-Nazi affiliations and violent activity.
- **Hammerskins Emblem**: Two crossed hammers representing a white supremacist skinhead group involved in hate music and propaganda.
- **Identitarian Movement Emblem**: A yellow circle with a black Greek lambda, symbolizing European ethno-nationalism and anti-immigration rhetoric.
- **Kolovrat**: An ancient Slavic sun symbol re-appropriated by white supremacist groups in Eastern Europe.
- **National Rebirth of Poland (NOP)**: Symbol of a far-right Polish nationalist group promoting xenophobic and anti-Semitic ideologies.
- **Popular Front Emblem**: Used by fascist-aligned organizations; typically combines nationalist imagery with militaristic symbols.

3.1.4 Task-Oriented Dataset Structuring

The dataset was structured to serve two complementary classification tasks:

- **Binary Classification (Symbol Detection):** All images containing any of the 22 Nazi-related symbols were labeled as *Nazi*, while the remaining images were labeled *non-Nazi*. The dataset includes approximately 2,300 positive and 99,000 negative images, resulting in a class ratio of 1:42.

- **Multi-Class Classification (Symbol Identification):** Only images from the 22 Nazi symbol categories were included. Each retained its class-specific label, totaling around 2,300 images. Due to severe class imbalance, symbol classes with very few samples were grouped into a new composite category labeled *neo-Nazi* (see Table 3.2), improving balance and model generalizability without sacrificing interpretability.

Class	**Images**
Black Sun	152
Flash and circle	13
Broken Sun Cross	21
The Happy Merchant	39
Hitler	228
Nazi salute	6
neo-Nazi	188
Yellow badge	4
Siegrune	311
SS Skull	257
Sturmabteilung emblem	11
Swastika	1070
Wolfsangel	38

(a) Regrouped symbol distribution

Class	**Images**
Atomwaffen Division	8
Blood & Honor emblem	25
Celtic Cross	51
Combat 18 emblem	6
Golden Dawn	19
Hammerskins emblem	12
Identitarian movement	11
Kolovrat	38
National Rebirth of Poland	10
Popular Front emblem	8

(b) neo-Nazi Symbol distribution

Table 3.2: Symbol distribution

All data collection and usage procedures were carried out in accordance with ethical research guidelines. No personal or sensitive identifiable content was included, and all symbols were used solely for academic purposes aimed at combating extremism.

3.2 Classification Models

To rigorously assess the efficacy of various ML methodologies in the detection and classification of Nazi symbols, we conducted experiments using a diverse set of models. This collection comprises both state of the art (SotA) DL architectures and traditional ML algorithms, enabling a comprehensive comparison across different computational paradigms and levels of model complexity.

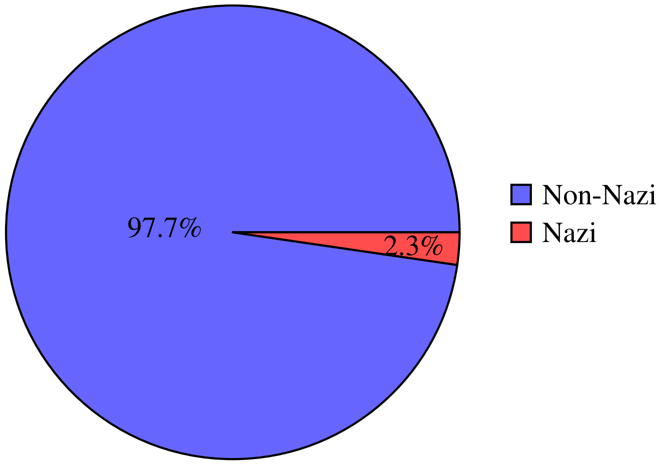


Figure 3.3: Pie chart showing binary classification dataset distribution.

3.2.1 You Only Look Once (YOLO)

YOLO is a real-time object detection framework that formulates detection as a unified regression problem, simultaneously predicting bounding boxes and class probabilities directly from full images. By processing the entire image in a single forward pass, YOLO achieves a favorable trade-off between inference speed and accuracy, rendering it highly suitable for scenarios demanding real-time processing. Although primarily designed for object detection, YOLO can be adapted for classification tasks by assigning the class label corresponding to the highest-confidence detection. In this study, we utilize the pretrained `YOLOv11-cls` classification model, which benefits from improved backbone architectures and refined label assignment strategies that enhance both robustness and computational efficiency.

3.2.2 Residual Network (ResNet)

ResNet is a CNN architecture that introduces residual learning through identity shortcut connections, thereby mitigating the vanishing gradient problem often encountered in training deep networks. This design facilitates the effective training of substantially deep architectures. The ResNet family has consistently demonstrated superior performance in image classification benchmarks. We specifically employ `ResNet34`, which offers a well-established balance between depth, computational cost, and classification accuracy. The `ResNet34` model used in this thesis is pretrained on the ImageNet dataset, enabling effective transfer learning for the target classification tasks.

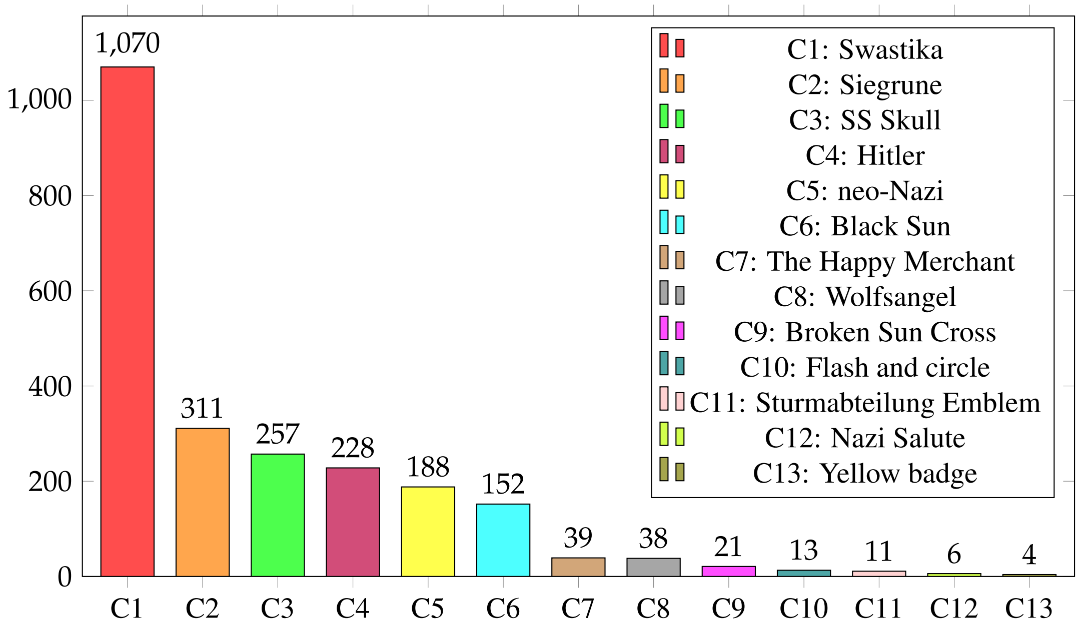


Figure 3.4: Dataset distribution of the multi-class classification task.

3.2.3 EfficientNet

EfficientNet represents a family of CNNs that leverage a compound scaling method to uniformly scale network depth, width, and resolution. This principled approach yields models that achieve SotA accuracy with significantly fewer parameters and computational resources. Due to its efficiency, EfficientNet is particularly well-suited for large-scale image classification tasks under constrained hardware conditions. In this thesis, we employ the `EfficientNet-B2` variant, which offers a favorable trade-off between model complexity and accuracy. It serves as a lightweight yet competitive alternative to deeper, more computationally intensive networks in our experiments. The `EfficientNet-B2` model used is pretrained on the ImageNet dataset, enabling transfer learning to improve performance on the target task.

3.2.4 Vision Transformer (ViT)

The ViT model innovatively adapts the transformer architecture, originally devised for Natual Language Processing (NLP), to vision tasks by segmenting images into fixed-size patches, which are then treated as a sequence of tokens. It eschews traditional convolutional operations entirely, instead relying on self-attention mechanisms to model long-range dependencies and capture global contextual information. ViT has demonstrated performance on par with or exceeding that of conventional CNNs, particularly when trained on large datasets.

In this thesis, we use the pretrained `vit_base_patch16_224` model to evaluate the capacity of transformer-based architectures to discern complex semantic patterns inherent in visual symbols.

OpenCLIP

OpenCLIP is an open-source reimplementation of CLIP, a multimodal vision-language model trained on extensive datasets of image-text pairs. By learning a shared embedding space for both modalities, OpenCLIP enables zero-shot classification through similarity comparison between image embeddings and text prompt embeddings. In this study, OpenCLIP is employed in two distinct modes: zero-shot inference and as a fixed feature extractor, providing a flexible and semantically enriched representation aligned with natural language descriptions.

3.2.5 Moondream

Moondream is a compact multimodal model optimized for low-resource vision-language tasks. Although not explicitly designed for image classification, it can be adapted for lightweight classification via prompt-based image-text similarity comparisons. Its strength lies in efficient on-device inference, making it particularly advantageous for deployment in computationally constrained environments such as mobile or embedded systems. We investigate Moondream as a potential candidate for these resource-limited classification scenarios.

OpenCLIP as Encoder with Classical ML Classifiers

To harness the semantic richness of OpenCLIP embeddings, we employ OpenCLIP as a frozen feature encoder, generating fixed-length vector representations for input images. These embeddings subsequently serve as input to a variety of traditional ML classifiers. This two-stage pipeline facilitates rapid experimentation with interpretable, computationally efficient models, especially beneficial for small datasets or when DL approaches incur excessive computational costs. A detailed summary of the key characteristics of the classification algorithms employed is presented in Table 3.3.

Classic Classifiers Used:

- **Logistic Regression (LR):** A linear probabilistic model well-suited for binary classification tasks.
- **Stochastic Gradient Descent (SGD) Classifier:** An efficient linear classifier trained using stochastic gradient descent, scalable to large datasets.

- **KNeighbors Classifier:** A classifier implementation of K-Nearest Neighbors (KNN), which is a non-parametric, instance-based learning algorithm that assigns labels based on the proximity of training examples in feature space.
- **Support Vector Classifier (SVC):** A margin-based classifier effective in high-dimensional feature spaces, leveraging kernel methods to handle non-linear separability.
- **Decision Tree Classifier:** A hierarchical model that partitions the feature space via threshold-based splits, enabling interpretable decision rules.
- **Bagging Classifier:** An ensemble approach that aggregates multiple decision trees trained on bootstrapped samples to reduce variance and improve stability.
- **Ada Boost Classifier:** A sequential ensemble method that iteratively focuses on misclassified samples to enhance overall predictive accuracy.
- **Gradient Boosting Classifier:** Constructs strong predictive models through stage-wise optimization of a differentiable loss function using weak learners.
- **Random Forest Classifier:** An ensemble of decision trees trained with random feature selection, providing robustness against overfitting and improved generalization.
- **Voting Classifier:** Combines the predictions of multiple base models (e.g., LR, Random Forest, SVC) using soft or hard voting strategies to enhance classification performance.

Classifier	Model Type	Linear	Ensemble	Interpretable	Scalable
Logistic Regression	Parametric	Yes	No	High	High
SGD Classifier	Parametric	Yes	No	Moderate	Very High
KNeighbors Classifier	Instance-based	No	No	Low	Low
SVC	Parametric	Yes	No	Moderate	High
Decision Tree Classifier	Non-parametric	No	No	High	Moderate
Bagging Classifier	Non-parametric	No	Yes	Moderate	High
AdaBoost Classifier	Non-parametric	No	Yes	Low to Moderate	Moderate
Gradient Boosting Classifier	Non-parametric	No	Yes	Low	Moderate
Random Forest Classifier	Non-parametric	No	Yes	Moderate	High
Voting Classifier	Hybrid	Depends on estimators	Yes	Low	High

Table 3.3: Summary of classification algorithms and their key characteristics

3.2.6 Notation for Classification Models

For brevity and consistency, the following shorthand notations will be used throughout the remainder of this thesis:

- **YOLO** refers to the `YOLOv11` model.
- **ResNet** refers to `ResNet34`.
- **EfficientNet** refers to `EfficientNet-B2`.
- **ViT** refers to `vit_base_patch16_224`.

These notations are adopted solely for readability and convenience within the scope of this thesis and do not imply reference to the entire family of model architectures.

3.3 Data Preprocessing

To ensure the reliability and consistency of model training, we implemented a comprehensive data preprocessing pipeline comprising resizing, normalization, and offline data augmentation. These preprocessing steps are critical to mitigating noise, simulating real-world variations, and facilitating efficient model convergence.

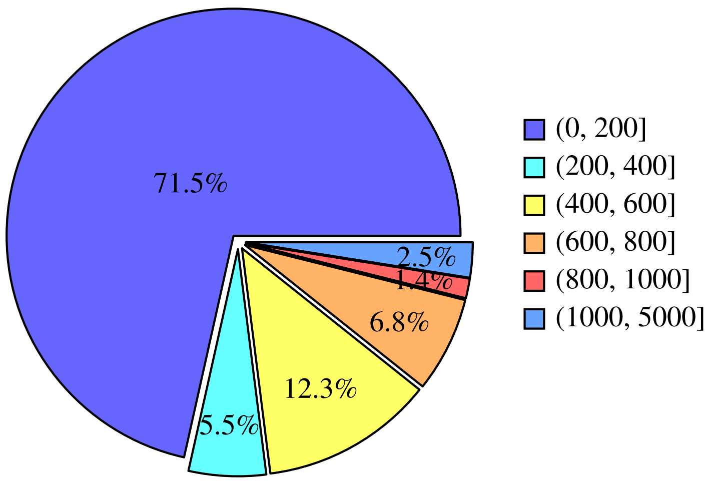


Figure 3.5: Distribution of image sizes in the dataset, grouped by pixel width and height. The majority of images fall into the $(0, 200]$ range.

- **Resizing:** All images were resized to a uniform resolution of 224×224 pixels. This choice was informed by two principal factors. First, as illustrated in Figure 3.5, analysis of the dataset revealed that approximately 71% of Nazi symbol images possess native dimensions smaller than 200×200 pixels, while only 5.5% fall within the 200×200 to 400×400 range. Standardizing to 224×224 pixels minimizes distortion and detail loss for the majority of images. Second, several evaluated architectures, including `vit_base_patch16_224`, require fixed input sizes due to their structural constraints. Adopting this resolution facilitates compatibility across models and ensures consistent comparative evaluation.

- **Normalization:** Pixel intensities were scaled to the range $[0, 1]$ on a per-image basis. This normalization stabilizes the optimization process and expedites convergence by ensuring consistent data scaling.
- **Data Augmentation:** Offline data augmentation was employed to artificially enhance the diversity of the training set and improve model generalization. Augmentations were applied probabilistically to each sample to simulate realistic variations encountered in operational contexts. The augmentation techniques included:
  - Conversion to grayscale, reducing reliance on color information.
  - Automatic contrast adjustment, normalizing image dynamic range.
  - Random horizontal and vertical flipping with a 50% probability, simulating orientation variability.
  - Random rotations within ±15°, introducing viewpoint variability.
  - Random shearing (vertically and horizontally within ±10°), simulating perspective distortions.
  - Random adjustments to hue (±15°), saturation (±63 units), and brightness (±38 units), increasing robustness to lighting and color shifts.
  - Random exposure modifications to simulate overexposure and underexposure conditions.
  - Gaussian blur applied randomly to emulate realistic image imperfections.
  - Sparse addition of random noise (0.1% probability) to enhance resilience against sensor noise and degradation.

  These augmentations reflect common distortions prevalent in real-world imagery, such as variable lighting, camera movement, compression artifacts, and historical degradation, which are particularly pertinent for Nazi symbol detection tasks.

All preprocessing operations were implemented using a combination of `Pillow` and `OpenCV` libraries during an offline stage. The processing pipeline applies resizing first, followed by augmentation and normalization, thereby ensuring computational efficiency and uniformity across all samples.

Examples illustrating the data preprocessing steps—comprising augmentation, resizing, and normalization—are depicted in Figure 3.6.

(a) Original image example.

(b) Augmented image example.

(c) Resized image example.

(d) Normalized image example.

Figure 3.6: Illustrative examples of data preprocessing steps.

3.4 Model Training

Each model employed in this study was trained with hyperparameter configurations carefully tailored to its architecture and the specific classification task at hand. To mitigate overfitting, early stopping was implemented for all DL models, monitoring validation loss with a patience of 10 epochs. The model checkpoint achieving the best validation performance was preserved for subsequent evaluation. A summary of the primary training hyperparameters is provided in Table 3.4.

ResNet was trained for 100 epochs using a batch size of 48 and an input resolution of 224×224 pixels. The binary classification tasks employed the Binary cross-entropy (BCE) loss, whereas multi-class problems utilized the categorical cross-entropy (CCE) loss function. Optimization was performed using the Adam algorithm with an initial learning rate of 1×10^{-3} and weight decay set to 1×10^{-5}. A 1-cycle learning rate scheduler was adopted to dynamically adjust the learning rate throughout training.

EfficientNet training spanned 100 epochs with a batch size of 48 and input size 224×224. For binary classification, the Nesterov momentum variant of Adam (NAdam) optimizer was used with a learning rate of 2×10^{-3}, no weight decay, and momentum decay of 0.004. For multi-class tasks, Adam optimizer was applied with a learning rate of 1×10^{-3}. The model's fully connected classification head was replaced with a linear layer matching the number of target classes.

ViT was trained for 100 epochs with batch size 48 and input resolution 224×224. The Adam optimizer with a learning rate of 2×10^{-5} was utilized. Loss functions were selected based on the task: BCE for binary classification and CCE for multi-class scenarios.

YOLO underwent training for 100 epochs, batch size 48, and input size 224×224. The SGD optimizer was employed with an initial learning rate of 0.01, momentum 0.9, and weight decay 5×10^{-4}. Correspondingly, BCE and CCE losses were applied for binary and multi-class classification, respectively.

OpenCLIP-based models leveraged OpenCLIP as a fixed feature extractor to obtain embeddings from resized input images (224×224). Subsequently, classical ML classifiers such as LR, SVC, and KNeighborsClassifier were trained on these embeddings using CPU resources. Zero-shot classification approaches with OpenCLIP and Moondream circumvented dataset-specific training entirely; classification was performed by computing cosine similarity between image and text embeddings, enabling direct inference without fine-tuning.

All training procedures were executed on an NVIDIA RTX 4060Ti GPU equipped with 16 GB of VRAM.

Learning rates were not unified across all models. Instead, they were selected based on architecture-specific best practices and empirical tuning. Transformer-based models such as ViT are sensitive to large gradient steps and thus require lower learning rates for stable convergence. Conversely, convolutional architectures like ResNet and EfficientNet can tolerate moderately higher learning rates. The values chosen reflect typical recommendations from the literature and were validated to ensure consistent training dynamics.

3.4.1 Prompt Design for Zero-Shot Classification

For zero-shot classification with OpenCLIP and Moondream, textual prompts were carefully designed to capture the distinctive visual and historical features of each Nazi symbol class. These prompts enable the models to relate image content to semantic concepts via image-text similarity scores or multimodal reasoning. Moondream's prompts take the form of analytical questions, eliciting binary ("Yes"/"No") answers that indicate the presence or absence of a symbol.

Model	Epochs	Batch Size	Input Size	Loss Function	Optimizer	Learning Rate
ResNet-34	100	48	224×224	BCE/CCE[1]	Adam	1e-3
EfficientNet-b2 (binary)	100	48	224×224	BCE	Nadam	2e-3
EfficientNet-b2 (multi)	100	48	224×224	CCE	Adam	1e-3
ViT	100	48	224×224	BCE/CCE	Adam	2e-5
YOLOv11	100	48	224×224	BCE/CCE	SGD	0.01
OpenCLIP + ML	–	–	224×224	–	–	–
Zero-shot (OpenCLIP, Moondream)[2]	–	–	–	–	–	–

[1] BCE = Binary Cross-Entropy, CCE = Categorical Cross-Entropy.
[2] Zero-shot models are not trained on the dataset; they use pretrained weights and inference directly.

Table 3.4: Summary of training hyperparameters used for each model.

Table 3.5 presents representative example prompts used for both OpenCLIP and Moondream models in this study. These examples illustrate how prompts differ in phrasing to leverage the specific strengths of each model.

Model	Example Prompt
OpenCLIP	"A black swastika symbol with arms bent at 90 degrees, typically rotated at a 45-degree angle, often shown on a red circular background or a white circle, used during World War II by Nazi Germany."
Moondream	"Analyse whether the image contains a black swastika symbol with arms bent at 90 degrees, typically rotated at a 45-degree angle, often shown on a red circular background or a white circle, used during World War II by Nazi Germany."

Table 3.5: Example prompts used for zero-shot classification with OpenCLIP and Moondream.

3.5 Evaluation Metrics

To quantitatively evaluate classification performance, several standard metrics commonly employed in image classification tasks were utilized. These metrics offer a multifaceted understanding of model predictive capabilities for both binary and multi-class problems.

Let the following definitions hold:

- TP: True Positives
- TN: True Negatives
- FP: False Positives

- FN: False Negatives

The metrics are formally defined as:

- **Accuracy**:
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$
Accuracy measures the proportion of correctly classified instances among all predictions and is most informative when class distributions are balanced.

- **Precision**:
$$\text{Precision} = \frac{TP}{TP + FP}$$
Precision quantifies the model's ability to avoid false positive errors, which is critical when the cost of false alarms is high.

- **Recall**:
$$\text{Recall} = \frac{TP}{TP + FN}$$
Recall assesses the model's effectiveness in identifying all relevant positive cases, particularly important in scenarios where missing positive instances carry severe consequences.

- **F1-Score**:
$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$
The F1-Score harmonizes precision and recall into a single metric, providing a balanced measure suitable for imbalanced class distributions.

- **Area Under the ROC Curve (AUC-ROC) and Average Precision (AP) (Optional for Binary Classification)**: These are threshold-independent metrics computed only for models supporting probabilistic predictions.
  - **AUC-ROC** evaluates a model's discriminative ability by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR) across various thresholds:
$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}$$
  - **AP** summarizes the precision-recall curve into a single scalar score by averaging precision across all recall levels.

These metrics were only reported for models that support probabilistic predictions. Classifiers such as `SVC`, `SGDClassifier`, and the `VotingClassifier`, as well as models like `Moondream`, were excluded from AUC-ROC and AP evaluations due to their lack of calibrated probability outputs. Similarly, the `OpenCLIP` model was excluded because it operates using cosine similarity in a high-dimensional embedding space and does not generate confidence scores or probabilistic predictions. As such, threshold-based metrics like AUC-ROC and AP are not applicable to its zero-shot, similarity-driven inference mechanism, and including them would not yield meaningful or interpretable results.

In the **multi-class setting**, AUC-ROC and AP were not employed due to their increased complexity and reduced interpretability when generalized using one-vs-rest or one-vs-one strategies. While macro- and micro-averaged variants exist, they often obscure class-specific behavior and are less suitable for this study's imbalanced, symbol-specific dataset. Therefore, macro-averaged precision, recall, and F1-score were adopted instead, ensuring equal contribution from all classes.

All evaluation metrics were reported per model under both binary and multi-class settings. For binary classification, accuracy, precision, recall, F1-score, and—where applicable—AUC-ROC and AP were reported. For multi-class classification, macro-averaged metrics were prioritized. Model selection was guided by validation loss, and final performance metrics were computed on held-out test sets.

4 Ethical Considerations

In conducting research on the classification of Nazi symbols in images, several critical ethical considerations must be acknowledged and addressed to ensure responsible and conscientious scientific practice.

- **Handling of Sensitive Content:** Nazi symbols are intrinsically linked to hate, violence, and genocide. Datasets containing such imagery must be managed with the utmost care to prevent inadvertent exposure or normalization of hateful content. Access to these datasets should be strictly controlled, and clear disclaimers must accompany any dissemination or use.
- **Purpose and Intent:** It is imperative to explicitly state that the objective of this research is to support the identification and moderation of harmful content, thereby preventing the proliferation of hate symbols. Transparency regarding the project's aims is essential to preclude any misinterpretation or misuse that could inadvertently promote such imagery.
- **Legal Compliance:** Given that many jurisdictions (notably Germany and Austria) impose legal restrictions on the display, distribution, and use of Nazi symbols, compliance with all relevant laws and regulations is mandatory throughout data collection, processing, and presentation phases.
- **Data Sourcing and Annotation:** All data utilized must be sourced from publicly available and legally authorized repositories. In instances where human annotation is required, measures should be implemented to minimize annotators' exposure to distressing content, including the provision of appropriate warnings and mental health resources.
- **Bias and Misclassification Risks:** Classification models may erroneously identify benign images as containing Nazi symbols or fail to recognize altered or subtle instances. Such inaccuracies can result in false accusations or failure to detect harmful content. Therefore, rigorous efforts to mitigate bias and enhance model robustness—such as employing diverse and balanced training datasets—are essential.
- **Dual-Use Concerns:** The development of automated systems capable of detecting Nazi symbols may be susceptible to misuse, including facilitating the creation of covert variants designed to evade detection or enabling unethical surveillance. These dual-use risks must be acknowledged, with appropriate safeguards instituted to limit deployment and distribution.

- **Privacy and Consent:** When datasets include identifiable individuals (for example, persons depicted wearing such symbols), careful anonymization and respect for privacy rights are mandatory. Ethical clearance and informed consent should be obtained where applicable to uphold data protection standards.
- **Transparency and Accountability:** The entirety of the research process—including dataset composition, methodological decisions, and known model limitations—should be thoroughly documented and communicated. This transparency encourages responsible use and provides guidance for future researchers to prevent potential misuse of classification systems developed for sensitive content.

5 Results

5.1 Binary Classification Results

5.1.1 Tabular Results

The results for the binary classification task are summarized in Table 5.1, which reports accuracy, precision, recall, and F1-score for each evaluated model, Area under the curve (AUC) and AP for applicable models.

Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**	**AUC**	**AP**
YOLO	0.995	0.793	0.932	0.857	0.999	0.913
ResNet	0.996	0.843	0.893	0.867	0.999	0.943
EfficientNet	0.993	0.772	0.761	0.767	0.993	0.780
ViT	0.997	0.910	0.893	0.901	0.999	0.962
Moondream	0.924	0.153	**0.995**	0.265	-	-
OpenCLIP	0.314	0.013	0.639	0.025	-	-
OpenCLIP + Logistic Regression	0.996	0.923	0.882	0.902	**1.000**	**0.964**
OpenCLIP + SGDClassifier	0.996	0.924	0.880	0.901	-	-
OpenCLIP + KNeighborsClassifier	0.996	0.895	0.936	0.915	0.999	0.934
OpenCLIP + SVC	**0.997**	0.948	0.929	**0.939**	-	-
OpenCLIP + DecisionTreeClassifier	0.987	0.674	0.742	0.706	0.993	0.625
OpenCLIP + BaggingClassifier	0.989	0.744	0.717	0.730	0.955	0.742
OpenCLIP + AdaBoostClassifier	0.994	0.901	0.791	0.842	0.998	0.925
OpenCLIP + GradientBoostingClassifier	0.993	0.879	0.801	0.838	0.998	0.919
OpenCLIP + RandomForestClassifier	0.981	**0.983**	0.097	0.177	0.999	0.895
OpenCLIP + VotingClassifier	0.996	0.945	0.873	0.908	-	-

Table 5.1: Binary classification performance across different models. The best scores for each metric are highlighted in bold.

5.1.2 Visual Comparison

To facilitate a clear visual comparison of model performance on the binary classification task, bar plots were generated for all selected models, including YOLO, ResNet, EfficientNet, ViT, Moondream, OpenCLIP, and OpenCLIP combined with SVC.

Figure 5.1, Figure 5.2, Figure 5.3, and Figure 5.4 depict the performance metrics, with the left side of each figure representing binary classification results. These visualizations highlight trade-offs among models and assist in identifying the most effective architectures.

For binary classification, ViT, YOLO, and OpenCLIP-based models paired with classical classifiers—particularly SVC and KNeighborsClassifier—demonstrated superior performance. Specifically, ViT and OpenCLIP + SVC achieved the highest F1-scores of 0.901 and 0.939, respectively. Conversely, Moondream and the base OpenCLIP encoder exhibited comparatively lower effectiveness.

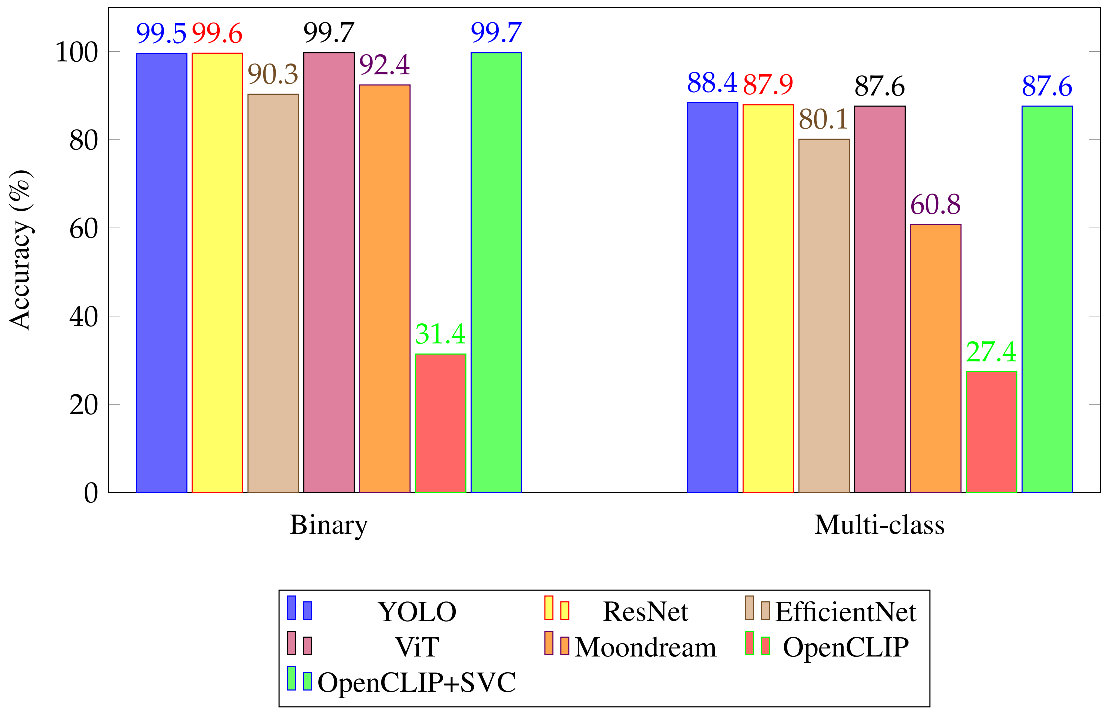


Figure 5.1: Comparison of accuracy scores for all models on binary (left) and multi-class (right) classification tasks.

5.1.3 Confusion Matrices

Confusion matrices for the binary classification task are presented in Figure 5.5. Each matrix corresponds to a model trained to distinguish images as either **Non-Nazi** or **Nazi**.

The evaluated models include ResNet, YOLO, EfficientNet, ViT, and OpenCLIP encoder combined with SVC.

These matrices provide detailed insight into each model's discriminative capabilities, where

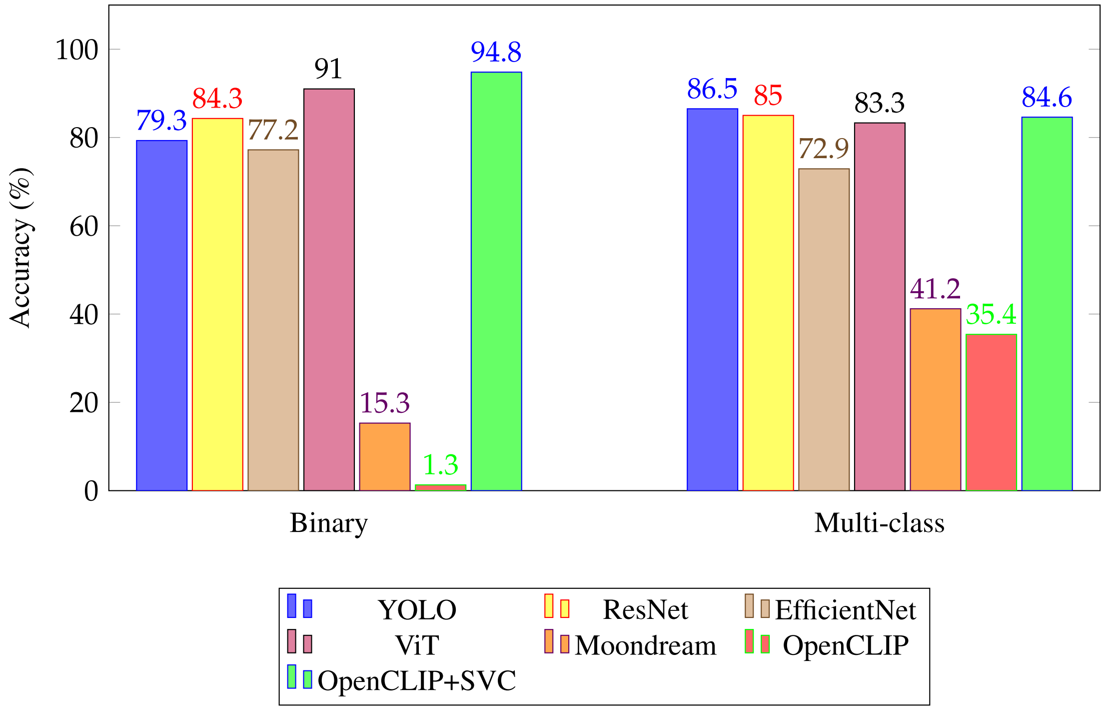


Figure 5.2: Comparison of precision scores for all models on binary (left) and multi-class (right) classification tasks.

a high count of true positives and true negatives indicates accurate classification, while false positives and false negatives reveal specific challenges such as visual similarity or low symbol clarity.

5.1.4 Receiver Operating Characteristic (ROC) and Precision-Recall (PR) Curves

To further characterize model performance under class imbalance, ROC and PR curves were plotted for the binary classification task.

Figure 5.6 illustrates the ROC curves, demonstrating the trade-off between true positive rate and false positive rate across different thresholds. The AUC values reported in Table 5.1 correspond with the visual trends observed.

Figure 5.7 presents the PR curves, which are particularly informative when evaluating classifiers under imbalanced conditions. The area under the PR curve (average precision) complements the AUC-ROC, emphasizing each model's ability to retrieve relevant positive samples effectively.

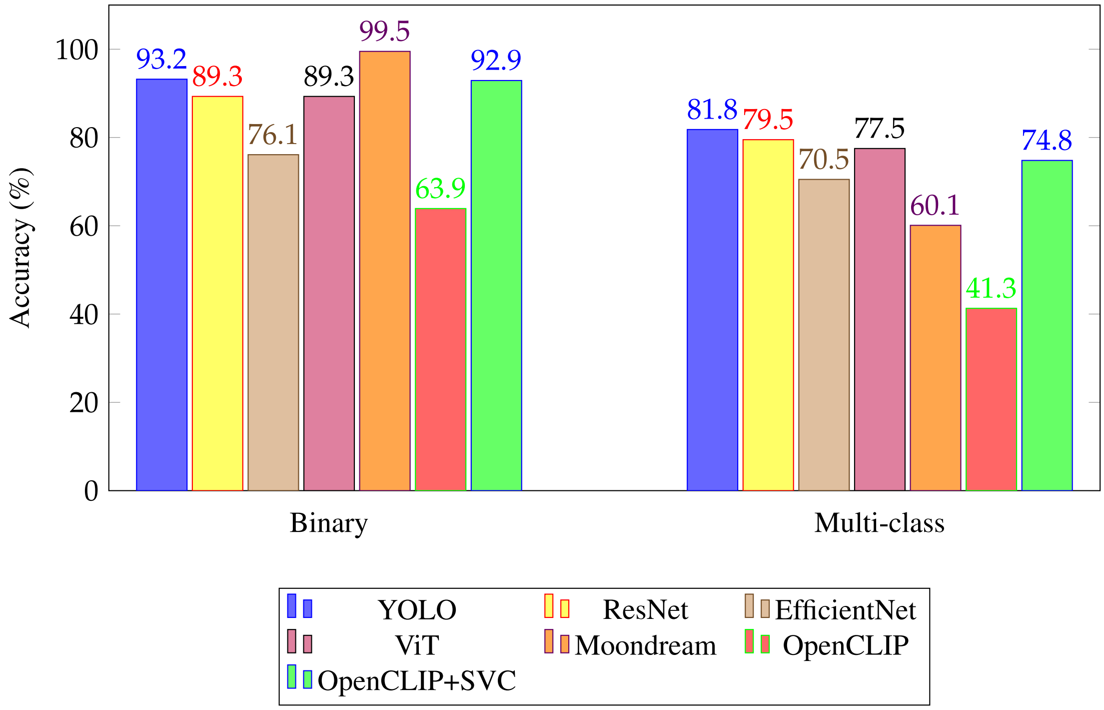


Figure 5.3: Comparison of recall scores for all models on binary (left) and multi-class (right) classification tasks.

5.1.5 Reduced-Symbol Set Experiment

To further evaluate model robustness, an additional experiment was conducted using a reduced set of Nazi symbols from the original dataset. This subset included the three most visually distinct symbols: the SS Skull, Siegrune, and Swastika.

The results of this reduced-symbol experiment are presented in Table 5.2, indicating the models' performance when focusing on a smaller, more visually distinct set of classes.

5.2 Multi-Class Classification Results

5.2.1 Tabular Results

The multi-class classification results are summarized in Table 5.3, reporting macro-averaged accuracy, precision, recall, and F1-score for each model.

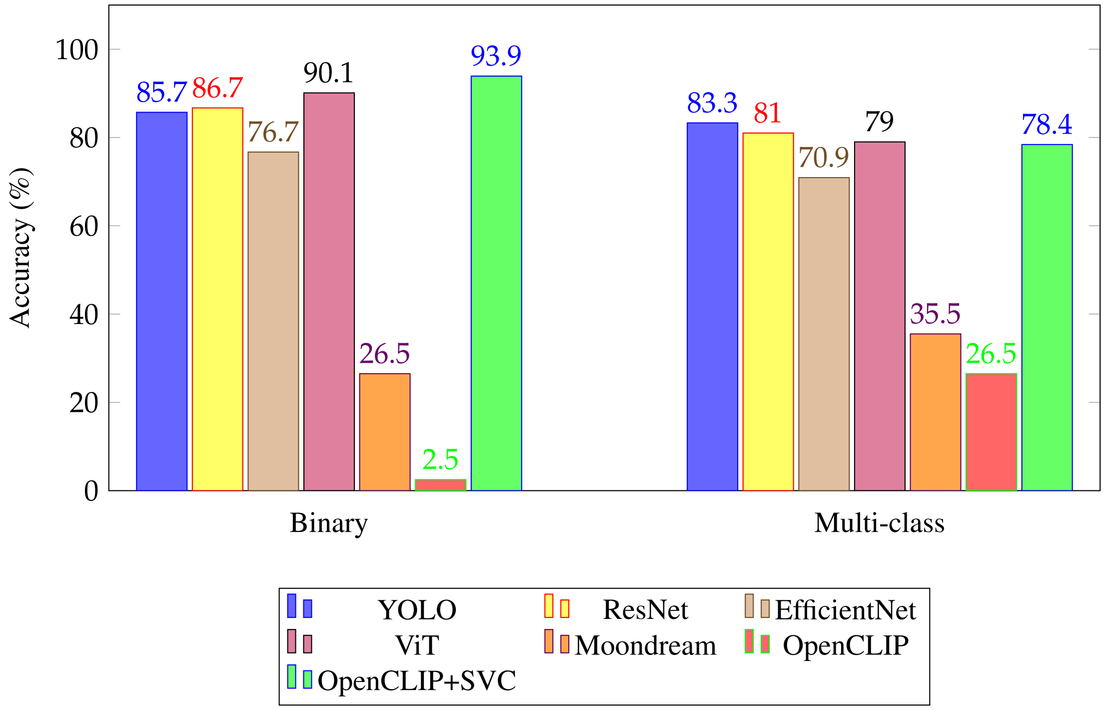


Figure 5.4: Comparison of F1-score values for all models on binary (left) and multi-class (right) classification tasks.

5.2.2 Visual Comparison

Figure 5.1, Figure 5.2, Figure 5.3, and Figure 5.4 also include the multi-class classification results, presented on the right side of each plot. These visualizations allow for direct comparison of model effectiveness across multiple symbol classes.

In this setting, YOLO consistently outperformed other models across all evaluation metrics, followed closely by ResNet and ViT, both of which demonstrated strong and consistent classification performance. Models such as Moondream and OpenCLIP showed less consistent results across different classes.

5.2.3 Confusion Matrices

Confusion matrices for the multi-class classification task are shown in the figures below. Each matrix corresponds to a model trained to classify images into one of thirteen symbol categories:

```
['Black Sun', 'Flash and circle', 'Broken Sun Cross', 'The
Happy Merchant', 'Hitler', 'Nazi salute', 'Yellow badge',
```

Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**
YOLO	0.994	0.792	0.568	0.661
ResNet	0.996	0.798	0.932	0.860
EfficientNet	0.987	0.667	0.744	0.703
ViT	**0.998**	0.972	0.926	**0.948**
Moondream	0.996	0.840	0.851	0.846
OpenCLIP	0.864	0.025	0.338	0.047
OpenCLIP + Logistic Regression	0.996	0.917	0.835	0.874
OpenCLIP + SGDClassifier	0.996	0.899	0.895	0.897
OpenCLIP + KNeighborsClassifier	**0.998**	0.895	**0.989**	0.940
OpenCLIP + SVC	**0.998**	**0.973**	0.900	0.935
OpenCLIP + DecisionTreeClassifier	0.985	0.514	0.635	0.568
OpenCLIP + BaggingClassifier	0.987	0.566	0.639	0.600
OpenCLIP + AdaBoostClassifier	0.994	0.869	0.742	0.800
OpenCLIP + GradientBoostingClassifier	0.993	0.837	0.708	0.767
OpenCLIP + RandomForestClassifier	0.988	0.959	0.209	0.344
OpenCLIP + VotingClassifier	0.997	0.964	0.833	0.894

Table 5.2: Binary classification performance across different models on reduced-symbol set. The best scores for each metric are highlighted in bold.

```
'neo-Nazi', 'Siegrune', 'SS Skull', 'Sturmabteilung emblem',
'Swastika', 'Wolfsangel']
```

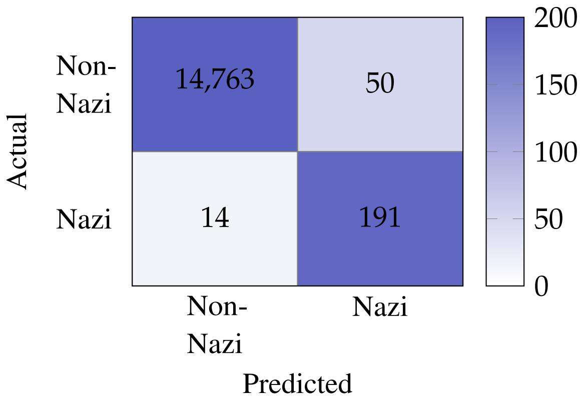
These matrices reveal class-specific performance, highlighting where misclassifications occur and identifying classes that are frequently confused. Such insights assist in understanding model limitations and potential areas for improvement.

- **ResNet:** Demonstrates strong performance on prominent symbols such as *Swastika* and *Siegrune*, with occasional confusion among visually similar classes.
- **YOLO:** Achieves high accuracy on frequent classes but struggles with rarer symbols like *Nazi salute* and *Broken Sun Cross*. Overall, it has the best performance across all classes.
- **EfficientNet:** Yields balanced performance but exhibits reduced confidence on ambiguous or underrepresented categories.
- **ViT:** Shows strong recognition for dominant classes, with some misclassifications on less frequent symbols.

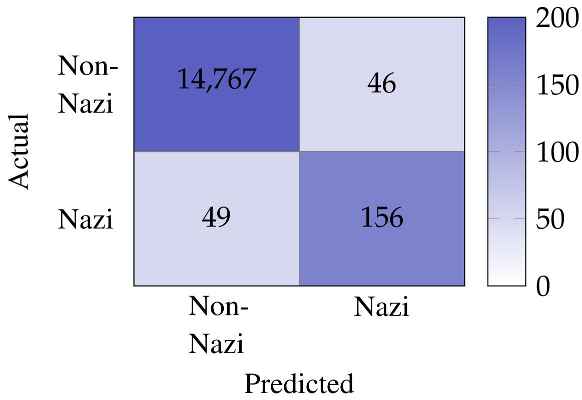
Model	Accuracy (Macro)	Precision (Macro)	Recall (Macro)	F1-score (Macro)
YOLO	**0.884**	**0.865**	**0.818**	**0.833**
ResNet	0.879	0.850	0.795	0.810
EfficientNet	0.801	0.729	0.705	0.709
ViT	0.876	0.833	0.775	0.790
Moondream	0.608	0.412	0.601	0.355
OpenCLIP	0.274	0.354	0.413	0.265
OpenCLIP + Logistic Regression	0.816	0.643	0.528	0.563
OpenCLIP + SGDClassifier	0.839	0.817	0.682	0.718
OpenCLIP + KNeighborsClassifier	0.863	0.818	0.750	0.775
OpenCLIP + SVC	0.876	0.846	0.748	0.784
OpenCLIP + DecisionTreeClassifier	0.573	0.222	0.168	0.174
OpenCLIP + BaggingClassifier	0.599	0.201	0.194	0.191
OpenCLIP + AdaBoostClassifier	0.668	0.503	0.329	0.363
OpenCLIP + GradientBoostingClassifier	0.841	0.738	0.677	0.703
OpenCLIP + RandomForestClassifier	0.573	0.246	0.169	0.170
OpenCLIP + VotingClassifier	0.836	0.757	0.557	0.609

Table 5.3: Multi-class classification performance across different models. The best scores for each metric are highlighted in bold.

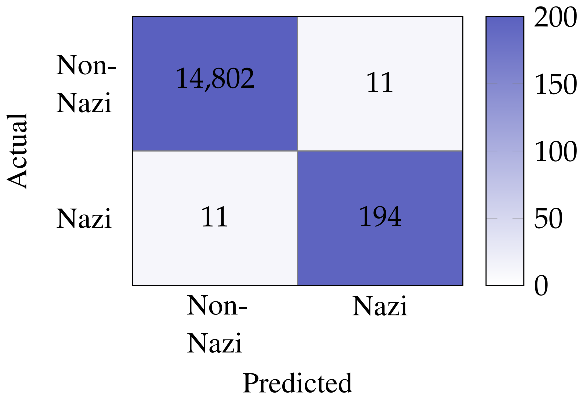
- **OpenCLIP encoder + SVC:** The text-image contrastive learning approach produces reasonable results, particularly for *Swastika*, but is sensitive to class imbalance.



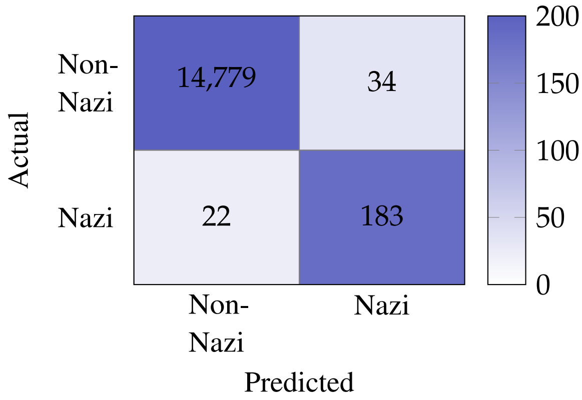
(a) Confusion matrix of YOLO.



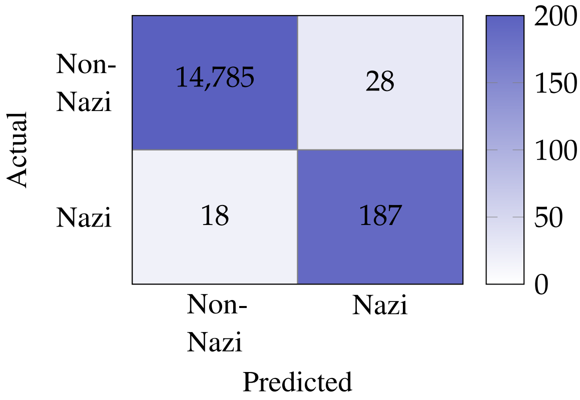
(b) Confusion matrix of EfficientNet.



(c) Confusion matrix of OpenCLIP encoder and SVC.



(d) Confusion matrix of ResNet.



(e) Confusion matrix of ViT.

Figure 5.5: Confusion matrices illustrating classification results of the binary task for each evaluated model. The matrices display true positive, true negative, false positive, and false negative counts for the classes “Nazi” and “Non-Nazi.”

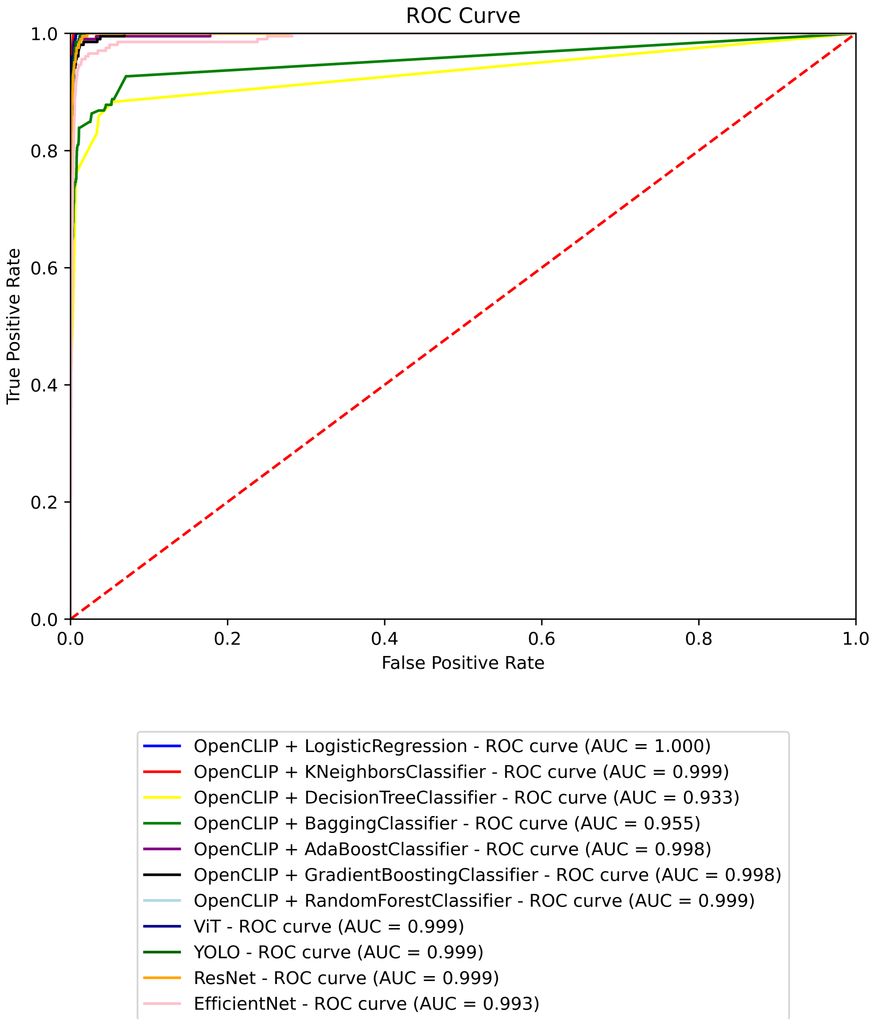


Figure 5.6: Receiver Operating Characteristic (ROC) curves for all models on the binary classification task, depicting the trade-off between true positive rate and false positive rate across classification thresholds.

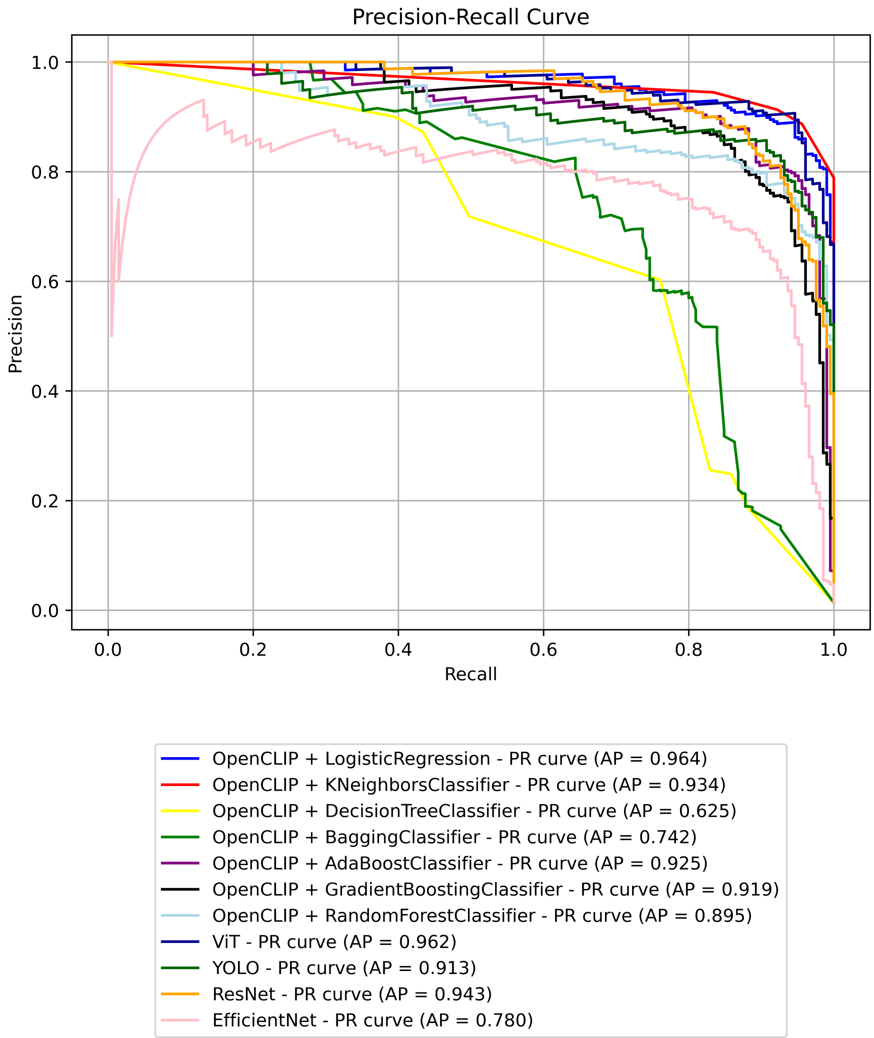


Figure 5.7: Precision-Recall (PR) curves for all models on the binary classification task, highlighting model performance under class imbalance conditions.

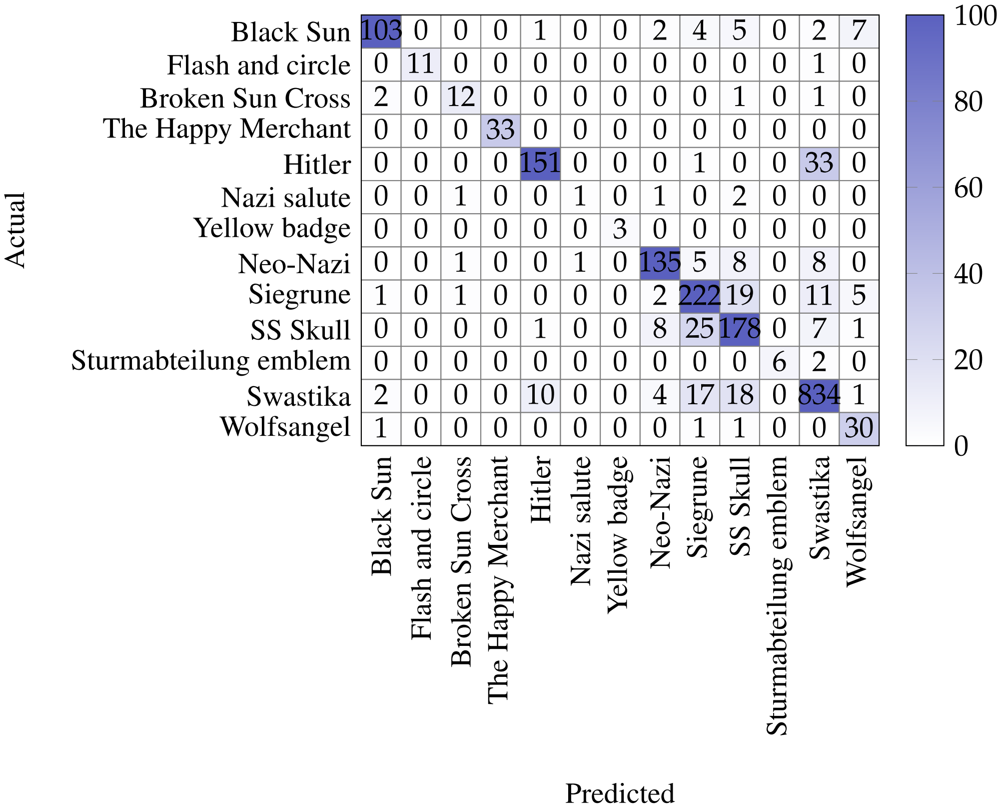


Figure 5.8: Confusion matrix for the binary classification task using YOLO. The values represent the number of samples in each cell.

Actual \ Predicted	Black Sun	Flash and circle	Broken Sun Cross	The Happy Merchant	Hitler	Nazi salute	Yellow badge	Neo-Nazi	Siegrune	SS Skull	Sturmabteilung emblem	Swastika	Wolfsangel
Black Sun	99	0	0	0	0	0	0	6	4	7	0	4	4
Flash and circle	0	9	0	0	0	0	0	0	0	0	1	2	0
Broken Sun Cross	0	0	9	1	0	0	0	2	0	2	0	2	0
The Happy Merchant	0	0	0	31	0	0	0	1	0	0	0	1	0
Hitler	2	0	1	0	140	0	0	1	2	8	0	28	3
Nazi salute	0	0	0	0	0	1	0	3	0	1	0	0	0
Yellow badge	0	0	0	0	0	0	3	0	0	0	0	0	0
Neo-Nazi	4	0	1	1	0	1	0	125	3	3	0	17	3
Siegrune	4	0	3	1	2	0	0	14	174	40	0	20	3
SS Skull	5	0	0	0	0	0	0	15	15	164	0	17	4
Sturmabteilung emblem	0	0	0	0	0	0	0	0	0	1	4	2	1
Swastika	9	1	0	5	31	0	0	20	17	18	2	780	3
Wolfsangel	7	0	0	0	0	0	0	1	2	1	0	3	19

Color scale: 0, 20, 40, 60, 80, 100

Figure 5.9: Confusion matrix for the binary classification task using EfficientNet. The values represent the number of samples in each cell.

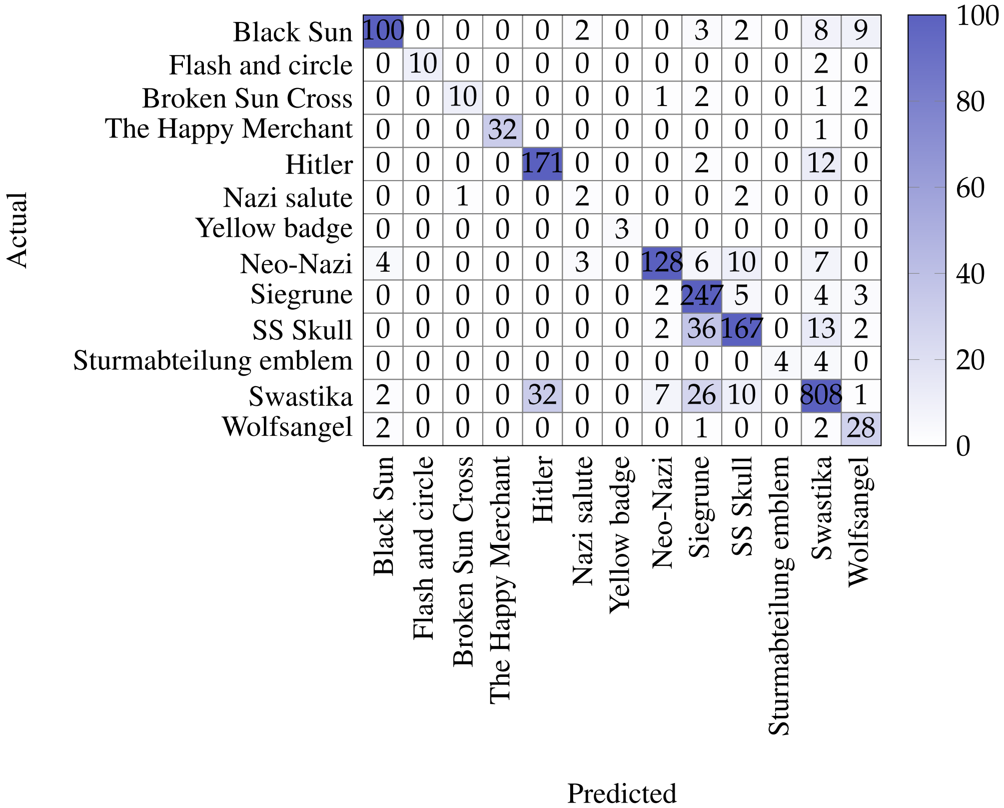


Figure 5.10: Confusion matrix for the binary classification task using ResNet. The values represent the number of samples in each cell.

Actual \ Predicted	Black Sun	Flash and circle	Broken Sun Cross	The Happy Merchant	Hitler	Nazi salute	Yellow badge	Neo-Nazi	Siegrune	SS Skull	Sturmabteilung emblem	Swastika	Wolfsangel
Black Sun	106	0	0	0	1	1	0	0	4	2	0	3	7
Flash and circle	0	11	0	0	0	0	0	0	0	0	0	1	0
Broken Sun Cross	3	0	6	0	0	0	0	1	2	2	0	2	0
The Happy Merchant	0	0	0	33	0	0	0	0	0	0	0	0	0
Hitler	0	0	0	0	175	0	0	0	1	0	0	9	0
Nazi salute	2	0	0	0	0	0	0	2	0	0	0	1	0
Yellow badge	0	0	0	0	0	0	3	0	0	0	0	0	0
Neo-Nazi	4	0	0	0	0	0	0	126	10	10	0	8	0
Siegrune	0	0	0	0	0	0	0	1	225	32	0	3	0
SS Skull	2	0	0	0	1	0	0	6	19	188	0	3	1
Sturmabteilung emblem	0	0	0	0	0	0	0	0	0	0	6	2	0
Swastika	3	0	0	0	36	0	0	4	23	23	0	797	0
Wolfsangel	3	0	0	0	0	0	0	1	1	1	0	0	27

Figure 5.11: Confusion matrix for the binary classification task using ViT. The values represent the number of samples in each cell.

Actual \ Predicted	Black Sun	Flash and circle	Broken Sun Cross	The Happy Merchant	Hitler	Nazi salute	Yellow badge	Neo-Nazi	Siegrune	SS Skull	Sturmabteilung emblem	Swastika	Wolfsangel
Black Sun	106	0	0	0	0	0	0	1	3	6	0	6	2
Flash and circle	0	10	0	0	0	0	0	0	0	0	0	2	0
Broken Sun Cross	3	0	8	0	0	0	0	1	1	0	0	3	0
The Happy Merchant	0	0	0	33	0	0	0	0	0	0	0	0	0
Hitler	0	0	0	0	163	0	0	0	1	1	0	20	0
Nazi salute	0	0	0	0	0	0	0	1	0	2	0	2	0
Yellow badge	0	0	0	0	0	0	3	0	0	0	0	0	0
Neo-Nazi	2	0	0	0	0	0	0	134	7	7	0	8	0
Siegrune	0	0	0	0	0	0	0	4	232	17	0	8	0
SS Skull	2	0	0	0	1	0	0	11	25	167	0	14	0
Sturmabteilung emblem	0	0	0	0	0	0	0	0	1	0	5	2	0
Swastika	3	0	0	0	24	0	0	3	23	10	0	823	0
Wolfsangel	7	0	0	0	0	0	0	1	1	2	0	2	20

Figure 5.12: Confusion matrix for the binary classification task using OpenCLIP encoder and SVC. The values represent the number of samples in each cell.

6 Discussion and Analysis

6.1 Binary Classification

The results presented in Table 5.1 reveal a wide range of performances across different models. Notably, classical ML classifiers leveraging OpenCLIP embeddings consistently ranked among the top performers. The best overall F1-score (0.939) was achieved by the **OpenCLIP + SVC** model, which also demonstrated high precision (0.948) and recall (0.929). Similarly, the **OpenCLIP + Logistic Rergression** model achieved the highest AUC score (1.000) and highest AP (0.964), along with a strong recall of 0.882 and an F1-score of 0.902, indicating a solid ability to detect Nazi symbols with relatively few false negatives.

Among DL models, both **ViT** and **YOLO** performed competitively. ViT achieved an F1-score of 0.901, the highest among all standalone neural models, while YOLO achieved an F1-score of 0.857 with a notably high recall of 0.932, highlighting its capacity to detect true positives effectively. ResNet also performed well, with an F1-score of 0.867, slightly outperforming EfficientNet (0.767).

nomalously, the **Moondream** model achieved an exceptionally high recall of 0.995 but suffered from very low precision (0.153), resulting in a modest F1-score of just 0.265. This suggests a strong tendency to overpredict the Nazi class, leading to a high number of false positives.Similarly, the **OpenCLIP zero-shot** model performed poorly, with a low F1-score of 0.025, highlighting the limitations of zero-shot classification without additional fine-tuning or classifier integration.

Several classical ensemble models also showed robust performance when paired with OpenCLIP embeddings. For instance, **OpenCLIP + VotingClassifier** reached an F1-score of 0.908 and precision of 0.945, while **OpenCLIP + AdaBoostClassifier** and **OpenCLIP + GradientBoostingClassifier** achieved F1-scores of 0.842 and 0.838, respectively. However, the **OpenCLIP + RandomForestClassifier** produced an unusually high precision (0.983) but very low recall (0.097), resulting in a poor F1-score (0.177), indicating that it was highly conservative in predicting the Nazi class.

Overall, models that combine OpenCLIP embeddings with classical classifiers—especially SVC, KNeighborsClassifier, and VotingClassifier—outperformed all other approaches in this binary classification task. Nonetheless, end-to-end deep learning models like ViT and YOLO remain competitive and demonstrate strong generalization capabilities, particularly when trained on high-resolution inputs and sufficient labeled data.

6.1.1 Binary Classification Confusion Matrix Analysis

Figure 5.5 presents the confusion matrices for the binary classification task, where the goal is to distinguish between *Nazi* and *Non-Nazi* images.

The OpenCLIP encoder combined with SVC achieves the best overall balance, producing the highest number of true positives (194) and true negatives (14,802), alongside the lowest false positives (11) and false negatives (11). This indicates a strong capability to correctly detect Nazi symbols while minimizing misclassifications.

YOLO shows excellent sensitivity, with the second lowest false negative count (14), meaning it rarely misses Nazi instances, but it has a relatively higher false positive rate (50), which might lead to more false alarms.

ResNet also performs well, showing a favorable trade-off between false positives (34) and false negatives (22), maintaining a good level of accuracy and reliability.

ViT provides balanced performance, with moderate false positives (28) and false negatives (18), showing robustness in both detecting Nazi symbols and correctly rejecting non-Nazi images.

EfficientNet, while demonstrating strong true negative results, has the highest false negatives (49), indicating that it misses more Nazi cases compared to the other models, which might limit its applicability when recall is critical.

Overall, these confusion matrices confirm that the OpenCLIP + SVC approach is the most effective for binary classification in this context, offering a strong balance between precision and recall essential for sensitive symbol detection.

6.1.2 Binary Classification ROC and PR Curve Analysis

The ROC curve results indicate that most models achieved excellent discriminative ability in the binary classification task, with several models reaching near-perfect AUC values close to 1.0. Notably, OpenCLIP combined with LR, KNeighborsClassifier, and Random Forest classifiers, along with ViT, YOLO, and ResNet, all demonstrated near-identical, near-optimal ROC performance (AUC $\approx$ 1.000), suggesting their strong capacity to separate Nazi and Non-Nazi classes.

Ensemble methods such as AdaBoost and Gradient Boosting classifiers paired with OpenCLIP embeddings also performed robustly (AUC $\approx$ 0.998), while EfficientNet's slightly lower AUC of 0.993 still represents a high-quality classification boundary. Bagging and Decision Tree classifiers showed comparatively lower AUCs (0.955 and 0.933), indicating reduced robustness and potential susceptibility to misclassification in borderline cases.

The AP values from the PR curves further underscore these findings, revealing that OpenCLIP + LR and ViT provide the best balance between precision and recall, critical in scenarios where false positives and false negatives carry significant consequences. The lower AP of the OpenCLIP + Decision Tree classifier (0.625) suggests that, despite reasonable AUC scores, this model struggles to maintain precision at higher recall levels, reflecting less reliable positive class predictions.

These combined results imply that embedding-based methods (OpenCLIP) paired with strong classifiers (LR, KNeighborsClassifier) or transformer architectures (ViT) are most effective for the task. In contrast, simpler or single-model approaches such as Decision Trees show limitations in practical use despite acceptable ROC metrics.

Only models that support probabilistic outputs are included in this analysis. Classifiers such as `SVC`, `SGDClassifier`, and the `VotingClassifier`, as well as models like `Moondream` and `OpenCLIP`, are omitted from ROC and PR curve evaluations, as discussed in Section 3.5.

6.1.3 Binary Classification with Reduced Nazi Symbol Classes

To further examine model robustness and adaptability, we conducted a binary classification experiment restricted to a reduced subset of Nazi symbols (*siegrune*, *swastika*, and *SS skull*) against the full set of non-Nazi images. The goal was to evaluate model performance when the semantic diversity of the positive class is reduced.

Despite the narrower scope, overall trends remained consistent with the full binary task (as showed in Table 5.2). Transformer-based architectures continued to perform strongly. ViT achieved excellent results with an F1-score of **0.948**, indicating its strong ability to generalize even with reduced symbol variety. Among OpenCLIP-based hybrid models, *OpenCLIP + SVC* achieved the best overall performance with an accuracy of **0.998**, precision of **0.973**, recall of **0.900**, and F1-score of **0.935**, while *OpenCLIP + KNeighborsClassifier* attained the highest recall at **0.989**.

Interestingly, YOLO underperformed in this setup (F1-score **0.661**) compared to its results on the full binary task, potentially due to its reliance on spatial object features that may be less effective when class variety is reduced. Conversely, models like Moondream and OpenCLIP hybrids maintained high performance, indicating that text-image aligned embeddings may still effectively capture distinguishing features in constrained settings.

As this is a focused experiment with a reduced symbol set, we limit evaluation to the most informative and widely supported metrics—accuracy, precision, recall, and F1-score. Metrics such as AUC and AP are omitted for simplicity and consistency, especially given that not all models provide calibrated probability outputs.

This experiment highlights that reducing the complexity of the positive class does not necessarily simplify the task uniformly for all models. While it benefits embedding-based methods and transformers, object detection models like YOLO may be affected due to loss of visual diversity.

6.2 Multi-Class Classification

Table 5.3 summarizes the performance of all evaluated models on the multi-class classification task. The **YOLO** model achieved the best performance across all macro-averaged metrics, with an accuracy of 0.884, precision of 0.865, recall of 0.818, and F1-score of 0.833. This superior performance likely stems from YOLO's capability to extract fine-grained spatial features in high-resolution images, combined with its efficient end-to-end object-centric architecture.

The **ResNet** and **ViT** models also demonstrated strong performance, with F1-scores of 0.810 and 0.790, respectively. ResNet maintained a slightly higher accuracy (0.879) than ViT (0.876), suggesting that deep convolutional features still offer competitive advantages. Meanwhile, ViT's transformer-based design effectively captured semantic dependencies, reinforcing its value for tasks involving subtle symbol variations.

In contrast, **EfficientNet** underperformed in this task, with an F1-score of 0.709 and a macro recall of 0.705, indicating difficulty in generalizing across all classes. The **Moondream** model also showed weak results (F1-score of 0.355), despite a moderately high recall (0.601), suggesting overprediction of certain classes.

The **OpenCLIP zero-shot** model performed poorly overall, with an accuracy of 0.274 and F1-score of 0.265. These results reinforce that zero-shot approaches struggle when applied to highly specialized tasks like Nazi symbol recognition without task-specific adaptation or supervised training.

When OpenCLIP embeddings were combined with traditional classifiers, performance varied widely. **OpenCLIP + KNeighborsClassifier** (F1-score: 0.775) and **OpenCLIP + SVC** (F1-score: 0.784) were the most effective, indicating that neighborhood-based and kernel-based methods are capable of partially exploiting OpenCLIP's semantic features. Similarly, **OpenCLIP + SGDClassifier** and **OpenCLIP + GradientBoostingClassifier** achieved solid F1-scores of 0.718 and 0.703, respectively.

Conversely, ensemble-based models such as **RandomForest**, **Bagging**, and **DecisionTreeClassifier** performed poorly, with F1-scores around or below 0.2. These results suggest that these models may not align well with the high-dimensional structure of OpenCLIP embeddings, potentially due to overfitting or limited ability to model fine-grained inter-class distinctions.

Overall, end-to-end models such as YOLO, ResNet, and ViT clearly outperform both zero-shot approaches and OpenCLIP embedding pipelines in the multi-class setting. These findings emphasize the importance of strong visual feature extraction and task-aligned architectures when handling classification tasks involving subtle visual differences across many categories.

6.2.1 Comparison with Binary Classification Performance

Comparing the results from the binary (Table 5.1) and multi-class (Table 5.3) classification tasks reveals consistent trends in model effectiveness, but also highlights some key differences driven by task complexity.

In both settings, **YOLO** and **ViT** emerged as top-performing DL models. However, their relative strengths became more pronounced in the binary task. For instance, ViT achieved an F1-score of 0.901 and YOLO reached 0.857 in the binary case, whereas their multi-class F1-

scores dropped to 0.790 and 0.833, respectively. This drop reflects the increased difficulty of distinguishing between a larger number of visually similar symbols.

The benefit of combining **OpenCLIP embeddings with classical classifiers** was more significant in the binary setting. In that task, models like **OpenCLIP + SVC** and **OpenCLIP + KNeighborsClassifier** achieved F1-scores of 0.939 and 0.915 respectively, outperforming nearly all other approaches. However, in the multi-class scenario, while still strong (F1-scores of 0.784 and 0.775 respectively), their advantage diminished, likely due to increased inter-class ambiguity and the limitations of traditional classifiers in modeling fine-grained boundaries across many classes.

Interestingly, some ensemble models (e.g., GradientBoosting, VotingClassifier) showed reasonable binary classification performance (F1-scores around 0.84–0.91), but generally degraded in the multi-class case (F1-scores below 0.71), possibly due to overfitting or insufficient class-separation power.

The **Moondream** and **OpenCLIP zero-shot** models performed poorly in both tasks, with Moondream showing a pathological tendency to predict the positive class in binary classification (perfect recall but extremely low precision), and generally low macro F1-scores in multi-class classification. These outcomes highlight the risks of deploying zero-shot or generative models in specialized symbol recognition tasks without task-specific training.

Overall, binary classification appears more forgiving, allowing classical classifiers with pre-trained embeddings to compete closely with end-to-end DL models. In contrast, the multi-class task places a heavier burden on precise visual discrimination and inter-class separability, where deep models like YOLO and ViT retain a clear edge. This underscores the importance of matching model complexity and architecture with task granularity and class diversity.

6.2.2 Multi-Class Classification Confusion Matrix Analysis

The confusion matrices for the multi-class classification task reveal distinct patterns of model performance across classes, reflecting varying degrees of class separation and misclassification.

ResNet shows strong true positive rates for high-frequency classes such as *Swastika* (808 correct predictions) and *Siegrune* (247 correct), but also notable confusion between visually similar classes. For example, *SS Skull* is frequently misclassified as *Siegrune* (36 instances), while *Siegrune* mislabels *SS Skull* less often (5 instances). This asymmetric confusion highlights the difficulty ResNet faces in discriminating fine-grained symbol variations. Low-frequency classes like *Nazi Salute* perform poorly, with very few correct predictions.

YOLO generally maintains competitive accuracy for dominant classes, correctly identifying 834 *Swastika* instances and 222 *Siegrune* samples. Confusion between *SS Skull* and *Siegrune* remains substantial but somewhat more balanced compared to ResNet, with 25 *SS Skull* instances misclassified as *Siegrune* and 19 misclassifications in the opposite direction. This indicates improved inter-class separation but ongoing challenges for these similar classes.

EfficientNet exhibits a somewhat different confusion profile, with lower misclassification of *SS Skull* as *Siegrune* (15 instances), but higher misclassification of *Siegrune* as *SS Skull* (40 instances), showing an inverse asymmetry compared to ResNet. The true positive count for *Siegrune* is comparatively lower (174), suggesting less confident classification in this region of the feature space. Low-frequency classes continue to be problematic, with *Nazi Salute* achieving only a single correct prediction.

ViT marginally improves performance for certain classes, such as *SS Skull* and *Hitler*, with 188 and 175 correct predictions respectively. Confusion between *SS Skull* and *Siegrune* remains, with 19 *SS Skull* samples misclassified as *Siegrune* and 32 in the reverse direction, indicating persistent challenges in distinguishing subtle differences despite the transformer architecture's strengths.

OpenCLIP Encoder combined with SVC achieves a balanced confusion pattern. It correctly identifies 232 *Siegrune* and 167 *SS Skull* instances. The confusion between *SS Skull* and *Siegrune* is distributed moderately (25 and 17 misclassifications respectively), showing a more symmetric error pattern compared to other models. This suggests better inter-class separation in this challenging pair, although some misclassification persists. Additionally, the model excels at maintaining tight clusters around true labels for high-frequency classes such as *Swastika* (823 correct) but struggles with low-frequency classes like *Nazi Salute*, which has zero correct predictions.

Overall, all models face difficulties with fine-grained classes like *SS Skull* and *Siegrune*, but OpenCLIP+SVC demonstrates a more balanced confusion profile. The results highlight the ongoing need to improve discrimination for visually similar symbols, especially for classes with limited training data.

7 Model Deployment and API Service

To facilitate real-world application and integration of the trained classification models, an API service was developed to serve the best-performing models from both the binary and multi-class classification tasks. The service was designed with a strong emphasis on modularity, portability, and usability.

Although the API was not deployed to an online hosting platform due to infrastructure constraints, it was fully implemented, containerized using Docker, and published to Docker Hub. This approach ensures reproducibility and allows collaborators (e.g., from the social sciences) or third-party researchers to easily run the service in local or cloud-based environments supporting Docker runtime.

7.1 System Architecture

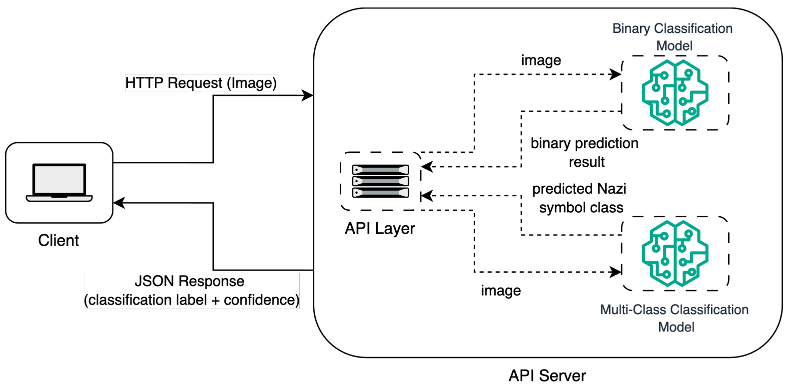


Figure 7.1: System architecture of the implemented classification API service.

Figure 7.1 illustrates the system architecture of the classification API service. Users submit an image file via an HTTP request, which is routed to one of three model pipelines depending on the specified task: binary classification, multi-class classification, or a two-stage hierarchical classification pipeline.

In the two-stage pipeline, the image is first passed to a binary classifier to determine whether it contains Nazi symbolism. If no such content is detected, the API returns a negative classification.

If Nazi-related content is identified, the image is subsequently forwarded to the multi-class classifier, which predicts the specific type(s) of Nazi symbol(s), such as swastikas, SS bolts, or eagles.

This cascaded design reduces computational overhead for irrelevant inputs and mirrors real-world content moderation workflows, where fine-grained analysis is only triggered upon initial positive screening.

The backend is implemented using `FastAPI`. All models are preloaded into memory at service startup, ensuring low-latency inference. The architecture is stateless and fully containerized, enabling straightforward horizontal scaling in distributed environments.

7.2 Model Integration and Routing Logic

The API exposes distinct endpoints for binary classification, multi-class classification, and the combined 2-layer pipeline. Instead of selecting the model dynamically based on request parameters, each endpoint corresponds to a specific inference path.

This design simplifies both the server-side routing and the client interface, enabling users to directly interact with the appropriate endpoint. Each endpoint internally handles preprocessing, model inference, and result formatting.

At service startup, all models are preloaded into memory. DL models implemented with PyTorch are serialized using `TorchScript` or the standard PyTorch save format. Classical ML models (e.g., logistic regression or SVC trained on OpenCLIP embeddings) are serialized using `joblib`. This preloading strategy reduces runtime latency and avoids repeated I/O overhead.

The modular architecture allows easy extension. New models or task-specific pipelines can be integrated by registering additional routes in the API codebase, without disrupting existing functionality.

Figure 7.2 summarizes the endpoint-to-model routing logic.

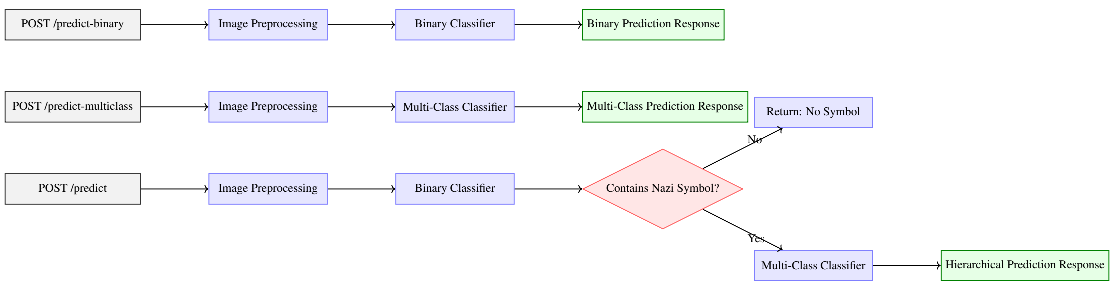


Figure 7.2: API endpoints mapped to model pipelines, including the 2-layer classification logic.

7.3 Continuous Integration and Deployment

The service is integrated with a GitHub Actions-based CI/CD pipeline, shown in Figure 7.3, which automates:

- Code quality checks and unit testing
- Docker image builds and tagging
- Generation and deployment of Sphinx-based documentation
- Publication of Docker images to Docker Hub

Although the API is not deployed to a public endpoint, the pipeline ensures that each commit to the main branch undergoes validation and build procedures, promoting maintainability. Users can reliably pull the latest version from Docker Hub and execute the service locally or in private cloud environments with minimal setup.

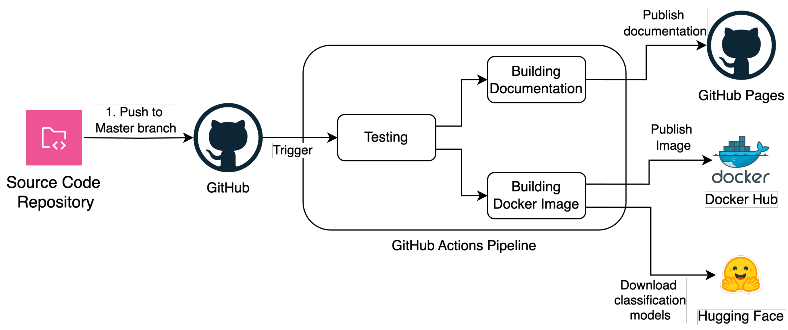


Figure 7.3: GitHub Actions CI/CD pipeline for Dockerized API service.

7.4 API Endpoints

Table 7.1 summarizes the available endpoints and their functionality.

The 2-layer endpoint encapsulates the binary–multi-class cascade, simplifying client-side logic and supporting integration into broader moderation or research pipelines.

All classification endpoints accept images as multipart form data and return structured JavaScript Object Notation (JSON) responses. These responses include predicted classes, confidence scores, and model metadata. The interactive Swagger UI at `/docs` facilitates testing and exploration.

Method	Endpoint	Description
`POST`	`/predict`	Runs the 2-layer pipeline: binary classification followed by multi-class prediction if Nazi content is detected. Threshold for symbol inclusion is 0.1.
`POST`	`/predict-binary`	Performs binary classification with associated confidence scores.
`POST`	`/predict-multiclass`	Performs multi-label classification of Nazi symbol types. Returns predictions above confidence threshold 0.1.
`GET`	`/health`	Health check endpoint.
`GET`	`/version`	Returns model and API version metadata.
`GET`	`/docs`	Serves OpenAPI documentation via Swagger UI.

Table 7.1: Summary of available API endpoints.

7.4.1 Example JSON Response

Figure 7.4 shows a sample JSON response for the binary classification endpoint. Figure 7.5 illustrates a multi-class classification response, while Figure 7.6 demonstrates the 2-layer classification response format.

```
[
  {
    "prediction": "nazi",
    "confidence": 0.965
  }
]
```

Figure 7.4: Sample JSON response for binary classification.

7.4.2 Thresholding vs. Top-k in Multi-label Classification

In the multi-class classification task, images may contain more than one Nazi symbol, constituting a multi-label problem. Instead of using a top-k strategy, which returns the k most confident classes regardless of absolute probability, a fixed confidence threshold of 0.1 was used to filter results.

This design offers two key advantages:

1. **Adaptability to Varying Label Counts**: The number of relevant symbols varies across

```
{
  "nazi_symbols": ["swastika", "siegrune", "black_sun"],
  "details": [
    {"label": "swastika", "confidence": 0.882},
    {"label": "siegrune", "confidence": 0.062},
    {"label": "black_sun", "confidence": 0.016}
  ]
}
```

Figure 7.5: Sample JSON response for multi-class classification.

```
{
  "containing_nazi_symbols": true,
  "prob": 0.97,
  "nazi_symbols": ["swastika", "siegrune", "black_sun"],
  "details": [
    {"label": "swastika", "confidence": 0.882},
    {"label": "siegrune", "confidence": 0.062},
    {"label": "black_sun", "confidence": 0.016}
  ]
}
```

Figure 7.6: Sample JSON response for 2-layer classification.

images. A threshold-based method allows the model to return only those classes exceeding the confidence threshold, thereby adapting to the input's true label count.

2. **Reduction of False Positives**: Low-probability predictions are filtered out, reducing the likelihood of returning irrelevant or incorrect classifications. This is particularly important in sensitive domains such as Nazi symbolism detection, where misclassification could have serious social and legal implications.

This approach provides more nuanced, context-aware predictions and aligns with the goal of responsible content moderation.

7.5 Extensibility and Future Deployment

The API service is architected for extensibility. Future capabilities—such as object detection for symbol localization, Optical character recognition (OCR) for embedded text, or detection of other hate symbols—can be integrated by adding new model pipelines and routes. The decoupled routing logic and memory preloading facilitate straightforward extensibility.

Although the service is not deployed online, it is fully containerized and ready for local or remote deployment. The accompanying GitHub repository includes:

- A `Dockerfile` and `pyproject.toml` for consistent runtime environments
- Instructions for local deployment using `docker run` or `docker-compose`
- Example requests (e.g., via `curl`) and a Postman collection
- GitHub Actions workflows for testing, documentation generation, and Docker publishing

This setup supports reproducibility and enables non-technical collaborators (e.g., from social sciences) to run the service locally without additional configuration.

For future deployment on public platforms, the system could be hosted on AWS, Google Cloud, or Azure. It is compatible with container orchestration (e.g., Kubernetes) or serverless deployment (e.g., AWS Lambda + API Gateway). Additional features—such as user authentication, request throttling, and secure logging—would be needed for production-grade deployment.

The modular and portable design ensures that the service can evolve alongside the research project and scale into production or collaborative use cases as required.

8 Conclusion and Future Work

8.1 Conclusion

This thesis presented a comprehensive study on the classification of Nazi symbols in images using a diverse set of models, including CNNs (ResNet, EfficientNet), transformer-based architectures (ViT), object detection models (YOLO), and multimodal embeddings from OpenCLIP. In addition to end-to-end DL approaches, we evaluated hybrid pipelines that combined OpenCLIP-generated image embeddings with classical ML classifiers.

Two classification settings were addressed: binary classification (Nazi vs. non-Nazi) and multi-class classification (13 Nazi-related symbol classes).

In the binary classification task, several models demonstrated exceptional performance. OpenCLIP combined with classical classifiers—such as SVC, LR, KNeighborsClassifier, and SGDClassifier—achieved high F1-scores (≥ 0.89), with some reaching near-perfect ROC AUC values (e.g., LR with AUC = 1.0). Transformer-based ViT and object detector YOLO also yielded strong results, balancing high AUC and AP. These findings indicate that both learned visual-semantic embeddings and end-to-end deep feature extractors are effective for distinguishing Nazi symbols from non-Nazi content. While some models such as OpenCLIP + Decision Tree or Bagging Classifier lagged in PR performance, the overall trend confirms the strength of hybrid pipelines and attention-based architectures in this classification setting.

However, models such as OpenCLIP + Decision Tree or Bagging Classifier demonstrated noticeably weaker performance, particularly in the PR space, indicating limited reliability in predicting the positive class under imbalanced conditions. While EfficientNet performed reasonably well in terms of AUC (0.993), its lower AP (0.780) suggests it may be less suited to high-precision tasks in this context.

In the multi-class setting, YOLO remained the strongest performer, followed closely by ViT. OpenCLIP alone did not perform well in zero-shot classification, but its results improved considerably when paired with supervised classifiers—though still trailing end-to-end deep models. These findings highlight that while OpenCLIP embeddings encode useful visual-semantic information, their effectiveness heavily depends on the downstream classifier and task supervision.

In summary, the experiments demonstrate that both end-to-end DL models and hybrid systems can be highly effective for classifying Nazi symbols. Nonetheless, model choice significantly affects performance depending on the metric of interest (e.g., AUC vs. AP), and success depends not only on feature representations but also on classifier robustness and sensitivity to data imbalance and symbol diversity.

8.2 Future Work

Building on the encouraging results of this study, several key directions for future work emerge:

- **Dataset Expansion and Balancing:** Increasing the volume and diversity of labeled images, especially for underrepresented or visually ambiguous symbol classes, would likely improve multi-class classification performance and generalizability. Including hard negatives could also enhance the robustness of binary classifiers.
- **Fine-Tuning Vision Backbones and Multimodal Training:** While OpenCLIP embeddings performed well with classical classifiers, further gains may be achieved through task-specific fine-tuning of the vision backbone or full multimodal retraining. This could improve class separability and address limitations observed in zero-shot settings.
- **Precision-Oriented Optimization:** Models with high AUC do not always translate to high precision under class imbalance. Future work could explore loss functions or calibration techniques that explicitly optimize for precision and recall, especially when false positives have severe consequences.
- **Explainability and Human Oversight:** Integrating explainable AI (XAI) techniques like Gradient-weighted Class Activation Mapping (Grad-CAM) [Sel+19] or SHapley Additive exPlanations (SHAP) [25] will enhance interpretability, allowing users to understand and verify predictions. This is critical in sensitive use cases such as archival documentation or legal review.
- **Contextual and Multimodal Learning:** Extending the classification pipeline to include metadata (e.g., captions, filenames, surrounding text) may improve classification of ambiguous images. Joint vision-language models or contextual classifiers could aid in more nuanced understanding.
- **Real-Time Deployment and Scalability:** Building on the containerized API service, future work should target online deployment with support for low-latency inference, scalability across cloud platforms, and compliance with data privacy standards.
- **User Feedback and Continuous Learning:** Introducing human-in-the-loop systems with user feedback can help refine classification boundaries over time, improve trustworthiness, and support use in dynamic or evolving symbol landscapes.

Overall, this thesis establishes a strong foundation for practical hate symbol recognition. The most promising avenues involve balancing high precision with explainability and integrating contextual and user-driven inputs to support real-world deployment in ethically sensitive environments.

Abbreviations

ViT Vision Transformer

LR Logistic Regression

CNN Convolutional Neural Network

ML Machine Learning

CI/CD Continuous Integration/Continuous Delivery

DL Deep Learning

VGGNet Visual Geometry Group Network

ZSL Zero-Shot Learning

CLIP Contrastive Language-Image Pre-training

RN Relation Network

AI Artifical Intelligence

BiLSTM Bidirectional Long Short-Term Memory

ADL Anti-Defamation League

SA Sturmabteilung

SS Schutzstaffel

NOP National Rebirth of Poland

YOLO You Only Look Once

ResNet Residual Network

SotA state of the art

NLP Natual Language Processing

KNN K-Nearest Neighbors

SVC Support Vector Classifier

SGD Stochastic Gradient Descent

BCE Binary cross-entropy

CCE categorical cross-entropy

NAdam Nesterov momentum variant of Adam

AUC-ROC Area Under the ROC Curve

TPR True Positive Rate

FPR False Positive Rate

ROC Receiver Operating Characteristic

AP Average Precision

PR Precision-Recall

AUC Area under the curve

API Application Programming Interface

JSON JavaScript Object Notation

OCR Optical character recognition

XAI explainable AI

SHAP SHapley Additive exPlanations

Grad-CAM Gradient-weighted Class Activation Mapping

List of Figures

List of Tables

Bibliography

[13] *Anti-Defamation League*. 1913. URL: https://www.adl.org (visited on 05/30/2025).

[25] *SHapley Additive exPlanations (SHAP)*. 2025. URL: https://shap.readthedocs.io/en/latest/ (visited on 05/30/2025).

[BA25] M. D. Belete and G. K. Alitasb. "Identification of hateful amharic language memes on facebook using deep learning algorithms." In: *Systems and Soft Computing* 7 (2025), p. 200258. ISSN: 2772-9419. DOI: https://doi.org/10.1016/j.sasc.2025.200258.

[BAN22] S. BANERJEE. *Animal Image Dataset (90 Different Animals)*. https://www.kaggle.com/datasets/iamsouravbanerjee/animal-image-dataset-90-different-animals. Open Source Dataset. 2022.

[Dee22] DeepNets. *Car vs Bike Classification Dataset*. https://www.kaggle.com/datasets/utkarshsaxenadn/car-vs-bike-classification-dataset. Open Source Dataset. 2022.

[DEL25] E. DELI. *SCOOBY DOO CLASSIFICATION DATASET*. https://www.kaggle.com/datasets/esmanurdeli/scooby-doo-classification-dataset. Open Source Dataset. 2025.

[Dos+21] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2021. arXiv: 2010.11929 [cs.CV].

[DTP19] S. Das Bhattacharjee, W. J. Tolone, and V. S. Paranjape. "Identifying malicious social media contents using multi-view Context-Aware active learning." In: *Future Generation Computer Systems* 100 (2019), pp. 365–379. ISSN: 0167-739X. DOI: https://doi.org/10.1016/j.future.2019.03.015.

[Geb+21] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. D. III, and K. Crawford. *Datasheets for Datasets*. 2021. arXiv: 1803.09010 [cs.DB].

[Ger23] Gerry. *100 Sports Image Classification*. https://www.kaggle.com/datasets/gpiosenka/sports-classification/data. Open Source Dataset. 2023.

[GIH21] K. GIHAN. *kids and adults detection*. https://www.kaggle.com/datasets/kaushigihanml/kids-and-adults-detection. Open Source Dataset. 2021.

[He624] He6aHbTeTTJL. *Swastiki_Net Dataset*. https://universe.roboflow.com/he6ahbtettjl/swastiki_net. Open Source Dataset. visited on 2025-05-19. June 2024.

[Ian+15] F. N. Iandola, A. Shen, P. Gao, and K. Keutzer. *DeepLogo: Hitting Logo Recognition with the Deep Neural Network Hammer*. 2015. arXiv: 1510.02131 [cs.CV].

[img23a] imgann. *swastika Dataset*. https://universe.roboflow.com/imgann/swastika-o7bde. Open Source Dataset. visited on 2025-05-19. Nov. 2023.

[img23b] imgann. *violent object Dataset*. https://universe.roboflow.com/imgann/violent-object. Open Source Dataset. visited on 2025-05-19. Nov. 2023.

[Inc23] S. D. Inc. *swastika_detect Dataset*. https://universe.roboflow.com/safety-drob-inc/swastika_detect. Open Source Dataset. visited on 2025-05-19. Aug. 2023.

[Kav22] Kavinski. *Swastika_Detection Dataset*. https://universe.roboflow.com/kavinski/swastika_detection. Open Source Dataset. visited on 2025-05-19. Apr. 2022.

[KSH12] A. Krizhevsky, I. Sutskever, and G. E. Hinton. "ImageNet Classification with Deep Convolutional Neural Networks." In: *Advances in Neural Information Processing Systems*. Ed. by F. Pereira, C. Burges, L. Bottou, and K. Weinberger. Vol. 25. Curran Associates, Inc., 2012.

[Lak21] R. Lakshmanamoorthy. *People Clothing Segmentation*. https://www.kaggle.com/datasets/rajkumarl/people-clothing-segmentation. Open Source Dataset. 2021.

[mar23] martin. *detect Dataset*. https://universe.roboflow.com/martin-wmeff/detect-lhuia. Open Source Dataset. visited on 2025-05-19. June 2023.

[Nec22] D. Nechaeva. *swastika Dataset*. https://universe.roboflow.com/darja-nechaeva/swastika-uw8xt. Open Source Dataset. visited on 2025-05-19. Apr. 2022.

[Rad+21] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. *Learning Transferable Visual Models From Natural Language Supervision*. 2021. arXiv: 2103.00020 [cs.CV].

[Sar25] S. Sarkar. *Low light animals*. 2025. DOI: 10.34740/KAGGLE/DSV/10692802.

[Scr+21] R. Scrivens, T. Wojciechowski, J. Freilich, S. Chermak, and R. Frank. "Comparing the Online Posting Behaviors of Violent and Non-Violent Right-Wing Extremists." In: *Terrorism and Political Violence* (Mar. 2021). DOI: 10.1080/09546553.2021.1891893.

[Sel+19] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization." In: *International Journal of Computer Vision* 128.2 (Oct. 2019), pp. 336–359. ISSN: 1573-1405. DOI: 10.1007/s11263-019-01228-7.

[SET22] K. SETH. *Fruits and Vegetables Image Recognition Dataset*. https://www.kaggle.com/datasets/kritikseth/fruit-and-vegetable-image-recognition/data. Open Source Dataset. 2022.

[Shu25] D. Shutterstock. *AI vs. Human-Generated Images*. https://www.kaggle.com/datasets/alessandrasala79/ai-vs-human-generated-dataset. Open Source Dataset. 2025.

[SS17] K. Shahriari and M. Shahriari. "IEEE standard review — Ethically aligned design: A vision for prioritizing human wellbeing with artificial intelligence and autonomous systems." In: *2017 IEEE Canada International Humanitarian Technology Conference (IHTC)*. 2017, pp. 197–201. DOI: 10.1109/IHTC.2017.8058187.

[Sun+18] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales. *Learning to Compare: Relation Network for Few-Shot Learning*. 2018. arXiv: 1711.06025 [cs.CV].

[SZ15] K. Simonyan and A. Zisserman. *Very Deep Convolutional Networks for Large-Scale Image Recognition*. 2015. arXiv: 1409.1556 [cs.CV].

[Tec24] V. I. of Technology Chennai. *Content moderation Dataset*. https://universe.roboflow.com/vellore-institute-of-technology-chennai-iwyte/content-moderation-fwj5b. Open Source Dataset. visited on 2025-05-19. June 2024.

[Tüz+17] A. Tüzkö, C. Herrmann, D. Manger, and J. Beyerer. *Open Set Logo Detection and Retrieval*. 2017. arXiv: 1710.10891 [cs.CV].

[Vol23] M. Volf. *symbols Dataset*. https://universe.roboflow.com/martin-volf-xdpby/symbols-w93jw. Open Source Dataset. visited on 2025-05-19. Feb. 2023.

[Wan+19] J. Wang, W. Min, S. Hou, S. Ma, Y. Zheng, H. Wang, and S. Jiang. *Logo-2K+: A Large-Scale Logo Dataset for Scalable Logo Classification*. 2019. arXiv: 1911.07924 [cs.CV].

[Xia+19] Y. Xian, C. H. Lampert, B. Schiele, and Z. Akata. “Zero-Shot Learning—A Comprehensive Evaluation of the Good, the Bad and the Ugly.” In: *IEEE Transactions on Pattern Analysis & Machine Intelligence* 41.09 (Sept. 2019), pp. 2251–2265. ISSN: 1939-3539. DOI: 10.1109/TPAMI.2018.2857768.

[Zha+24] J. Zhang, M. Cao, X. Yang, K. Jiang, and Y. Li. *DiffCLIP: Few-shot Language-driven Multimodal Classifier*. 2024. arXiv: 2412.07119 [cs.CV].